OmniCode: Chapter 3: Track 1 - Abstract Syntax Tree & Parser

Lexical Tokens, Grammar Rules, AST Entities & Asides

CHAPTER IDENTIFIER
CH 03

NOTEBOOKLM WORD COUNT
18,326 words

CATALOGED CHAPTER FILES
21 files (4,552 lines)

GENERATION DATE
2026-10-16 20:35:49

EPISTEMIC COVENANT BEDROCK & IMMUTABLE LANDMARKS

"And he gave unto Moses... two tables of testimony, tables of stone, written with the finger of God." — Exodus 31:18

"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house." — Proverbs 24:27

"Remove not the ancient landmark, which thy fathers have set." — Proverbs 22:28

GOOGLE NOTEBOOKLM GROUNDING FOCUS & LEARNING PROMPT

- **Chapter Focus:** *"Explain how the AST preserves first-class intent asides."*
- **Architectural Rule:** All definitions conform to 4-block structure & spatial ruler.

CHAPTER MANIFEST & INDEX (21 FILES)

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
1	**<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/README.md**	MD	150	991	7.7
2	**<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/mod.rs**	RS	54	289	2.5
3	**<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/universal/char_stream.rs**	RS	358	1,246	11.3
4	**<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/universal/cursor.rs**	RS	324	1,248	12.6
5	**<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/universal/mod.rs**	RS	20	95	0.8
6	**<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/syntactic/grammar.rs**	RS	599	1,985	24.3
7	**<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/syntactic/mod.rs**	RS	20	93	0.8
8	**<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/syntactic/parser.rs**	RS	203	935	8.7
9	**<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/lexical/lexer.rs**	RS	678	2,858	32.0
10	**<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/lexical/mod.rs**	RS	20	95	0.8
11	**<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/lexical/tokens.rs**	RS	443	1,748	16.4
12	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/README.md**	MD	229	1,814	13.4
13	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/mod.rs**	RS	30	102	1.1
14	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/nodes/boundaries.rs**	RS	108	408	3.7
15	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/nodes/cardinal.rs**	RS	96	420	3.7
16	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/nodes/mod.rs**	RS	328	765	11.8
17	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/nodes/ontological.rs**	RS	235	862	7.2
18	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/nodes/operational.rs**	RS	158	564	5.2
19	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/nodes/primitives.rs**	RS	125	445	4.1
20	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/nodes/relations.rs**	RS	169	594	5.3
21	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/nodes/structural.rs**	RS	205	769	6.3

<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/README.md MD 150 LINES 991 WORDS 7.7 KB

```
# OmniCode Parser Subsystem & Ingestion Library

> *"And he gave unto Moses, when he had made an end of communing with him upon mount Sinai, two tables of testimony, tables of stone, written with the finger of God."*
> — **Exodus 31:18**
>
> *"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house."*
> — **Proverbs 24:27**
>
> *"Remove not the ancient landmark, which thy fathers have set."*
> — **Proverbs 22:28**

---

## 1. Architectural Philosophy: Why This Subsystem Is a Pure Library

The `parser` subsystem in **OmniCode Track 1 (Host Engine & Virtual Machine)** is not an executable application or monolithic script. It is an **autonomous, stateless, and mathematically pure software library**.

### Core Library Invariants

1. **Zero Side Effects**: The ingestion pipeline has no ambient state, initiates no hardware I/O, does not access disk or network directly, and runs identically on bare metal, hosted Linux/macOS/Windows, or embedded inside WebAssembly.
2. **Deterministic Transduction**: Given the identical UTF-8 source reel string, the pipeline is guaranteed to yield the exact same `Token` stream and typed Abstract Syntax Tree (`ASTNode`).
3. **Universal Consumption**: This library serves as the shared front-end across the entire OmniCode ecosystem:
   * **The Assembler (`assembler.rs`)**: Tokenizes declarative machine tapes and physical memory bindings.
   * **The Language Server & Hover Engine (`hover.rs`)**: Performs real-time syntax highlighting, boundary matching, and token span lookup.
   * **The Self-Healing Restorator (`restorator.rs`)**: Intercepts diagnostic failpoints to repair malformed source reels.
   * **The AI Semantic Transducer (`AITransducer`)**: Formulates and validates synthesized OmniCode blocks.
   * **The Virtual Machine Evaluator (`evaluator.rs`)**: Executes the typed AST nodes.

---

## 2. Subsystem Directory Topology

The library is organized into three clean layers of abstraction, separating universal streaming mechanics from custom OmniCode linguistic morphology:

```
src/parser/
├── README.md                    <-- Architectural Manifesto & Subsystem Documentation
├── mod.rs                       <-- Library Facade & 100% Backward-Compatible Re-exports
│
├── universal/                   <-- Pure Stream Navigation & Coordinate Tracking (Language-Agnostic)
│   ├── mod.rs                   <-- Re-exports universal cursors and checkpoints
│   ├── char_stream.rs           <-- Universal Character Stream Cursor (CharCursor, CharCheckpoint)
│   └── cursor.rs                <-- Universal Token Stream Cursor (TokenCursor, Expectation Guards)
│
├── lexical/                     <-- Lexical Inscription Subsystem (Morpheme Level)
│   ├── mod.rs                   <-- Re-exports token taxonomy and lexer
│   ├── tokens.rs                <-- Inscribed Token Vocabulary, TokenType (10 categories), Spans
│   └── lexer.rs                 <-- OmniCode Morphemic Scanner, Compounding & Intent Asides
│
└── syntactic/                   <-- Syntactic & Grammar Subsystem (AST & Boundary Level)
    ├── mod.rs                   <-- Re-exports grammar engine and parser facade
    ├── grammar.rs               <-- 6 Ranked Boundaries & 6-Level Precedence Cascade (GrammarEngine)
    └── parser.rs                <-- Recursive Descent Parser Facade Coordinator (Parser)
```

---

3. Subsystem Breakdown

A. `universal/` — Stream Navigation Machinery

Contains reusable cursors that operate on pure linear sequences without knowledge of OmniCode's grammar rules:

* **`CharCursor` (`char_stream.rs`)**:
  * 1-indexed `(line, col, byte_offset)` coordinate tracking.
  * CRLF normalization (`\r\n` and `\n` treated as a single line step).
  * Zero-allocation pattern matching (`match_str`) directly against character slices.
```

```
    * Speculative checkpointing and rollback (`CharCheckpoint`).
    * Universal accumulation primitives (`consume_while`).
  * **`TokenCursor` (`cursor.rs`)**:
    * Safe lookahead past EOF without panic (`peek()`, `peek_at(offset)`).
    * Saturating boundary depth counters (`bracket_depth`, `brace_depth`, `angle_depth`).
    * Structured diagnostic failpoint recording upon expectation mismatches.

### B. `lexical/` — Lexical Inscription Subsystem

Translates raw character reels into inscribed tokens of stone:

* **`tokens.rs`**:
  * Complete 10-category ontological hierarchy (`TokenType`):
    1. Delimiters & First-Class Asides (`[`, `]`, `{`, `}`, `<`, `>`, `Aside`)
    2. Grammatical Pronouns (`I`, `you`, `it`, `they`, `myself`, etc.)
    3. Polarities & Casts (`+Word`, `-Word`, `[+]`, `[-]`, `<+>`, `<->`, `{+}`, `{-}`)
    4. Directional Vectors (`-->`, `<--`, `^--`, `v--`, `/^`, `\v`, `/-^`, `\-v`)
    5. Relations of Being & Transduction (`=`, `!=`, `~=`, `<=>`, `<`, `>`, `<=`, `>=`)
    6. Connectives & Hardware Locators (`!`, `::`, `::@`, `|`, `--`, `&`)
    7. Spatial Locators & Ranges (`@`, `..`, `%`)
    8. Cardinal Classification Clauses (`WHO`, `WHAT`, `WHEN`, `WHERE`, `WHY`, `HOW`)
    9. SVO Arithmetic Keywords (`plus`, `minus`, `times`, `div`)
    10. Inferential & Literals (`?`, `+1`, `-1`, `0`, `NULL`, `0x...`, `'A'`, `"..."`)
  * Morphological predicates: `is_boundary()`, `is_relational()`, `is_vector()`, `is_hardware_locator()`, `is_state_prime()`.
* **`lexer.rs`**:
  * Compounding engine fusing base characters into multidimensional operators.
  * First-class Intent Aside extraction: preserves `( ... )` with nested depth tracking.
  * State-prime normalization (`Character'`, ```Character` ```) designating causal step $S \to S'$.

### C. `syntactic/` — Syntactic Grammar Subsystem

Constructs the typed Abstract Syntax Tree:

* **`grammar.rs` (`GrammarEngine`)**:
  * **The 6 Ranked Morphological Boundaries**:
    * Rank 1 `< ... >`: Invariant Axiom (Absolute Immutable Law)
    * Rank 2 `[ ... ]`: Denotation (Concrete State Container & Silicon Evaluation Block)
    * Rank 3 `{ ... }`: Connotation (Context, Environment & Payload Attributes)
    * Rank 4 `[ ... }`: Inside-Out Emergence (Output Projection to External Universe)
    * Rank 5 `{ ... ]`: Outside-In Imposition (Input Capture from External Universe)
    * Rank 6 `( ... )`: Intent Aside (First-Class Cognitive Teleology & Rationale)
  * **Downward Precedence Cascade (Flow Down)**:
    1. Caveats (`!`) and Connectives (`::`, `&`, `|`, `:`, `@`, `::@`)
    2. Directional Vectors (`-->`, `<--`, `^--`, `v--`, quadrant & scale vectors)
    3. Being Relations (`=`, `!=`, `~=`, `<=>`, `<`, `>`, `<=`, `>=`)
    4. SVO Arithmetic (`plus`, `minus`, `times`, `div`)
    5. Unary Operations (`--`, prefix vectors)
    6. Primary Units & Delimiters
  * **Upward Teleological Cascade (Flow Back Up)**:
    Attaches following Intent Asides `( ... )` to statements and boundary blocks.
* **`parser.rs` (`Parser`)**:
  * Public facade coordinating the `TokenCursor` and `GrammarEngine`.
  * Manages diagnostic failpoint intercepts and provides top-level reel parsing.

---

## 4. End-to-End Ingestion Lifecycle

```mermaid
graph TD
    A["Source Text Reel (UTF-8)"] --> B["CharCursor (universal/char_stream.rs)"]
    B --> C["Lexer (lexical/lexer.rs)"]
    C --> D["Token Stream + Spans (lexical/tokens.rs)"]
    D --> E["TokenCursor (universal/cursor.rs)"]
    E --> F["GrammarEngine (syntactic/grammar.rs)"]
    F --> G["Abstract Syntax Tree (ASTNode)"]
    G --> H["Assembler & Evaluator Runtime"]
```

---

5. Public API & Re-exports

For maximum ergonomic simplicity and 100% backward compatibility, `src/parser/mod.rs` re-exports the primary types at the crate root:

```rust
```
```

```
use crate::parser::{
    CharCursor, TokenCursor,
    Lexer, Token, TokenType, Span, SourceLocation,
    Parser, GrammarEngine,
};
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/mod.rs RS 54 LINES 289 WORDS 2.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Parser Subsystem Identity, Architecture Manifest & Re-exports )
// ============================================================================
//! # OmniCode Parser Subsystem & Ingestion Library (`parser`)
//!
//! The parser subsystem is a pure, stateless software library responsible for
//! transcribing raw source text reels into validated Abstract Syntax Trees (AST).
//!
//! ## Scriptural Grounding:
//! - **Exodus 31:18**: *"And he gave unto Moses, when he had made an end of communing
//!   with him upon mount Sinai, two tables of testimony, tables of stone, written with
//!   the finger of God."*
//! - **Proverbs 24:27**: *"Prepare thy work without, and make it fit for thyself in
//!   the field; and afterwards build thine house."*
//! - **Proverbs 22:28**: *"Remove not the ancient landmark, which thy fathers have set."*
//!
//! ## Subsystem Topology:
//! - [`universal`]: Language-agnostic Unicode character and token stream cursors.
//! - [`lexical`]: Ontological token taxonomy, morpheme compounding, and lexer.
//! - [`syntactic`]: Morphological boundary parser and dual-vector precedence cascade.

pub mod universal;
pub mod lexical;
pub mod syntactic;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Public Library Re-exports & Backward Compatibility Layer )
// ============================================================================
pub use universal::{CharCheckpoint, CharCursor, TokenCursor};
pub use lexical::{Lexer, SourceLocation, Span, Token, TokenType};
pub use syntactic::{GrammarEngine, Parser};

// Aliases preserving 100% backward compatibility for internal imports
pub use lexical::tokens;
pub use lexical::lexer;
pub use universal::cursor;
pub use universal::char_stream;
pub use syntactic::grammar;
pub use syntactic::parser;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Parser Subsystem Core Coordination )
// ============================================================================
// Parser library components are actively utilized across compilation and execution passes.

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Verification & Module Status )
// ============================================================================
// Parser subsystem verified, modularized, and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/universal/char_stream.rs | RS | 358 LINES | 1246 WORDS | 11.3 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal Character Cursor, Source Reel Traversal & Coordinate Tracking )
// ============================================================================
//! # Universal Character Stream Cursor (`char_stream.rs`)
//!
//! Provides pure, reusable Unicode character-stream navigation over OmniCode source reels.
//! Operates independently of grammatical syntax rules, providing deterministic cursor
//! advancement, coordinate tracking (1-indexed lines and columns), zero-allocation pattern
//! matching, and speculative backtracking.
//!
//! ## Scriptural Grounding:
//! - **Exodus 31:18**: *"And he gave unto Moses, when he had made an end of communing
//!   with him upon mount Sinai, two tables of testimony, tables of stone, written with
//!   the finger of God."*
//!   The character cursor reads the inscribed glyphs from the physical stone reel.
//! - **Proverbs 24:27**: *"Prepare thy work without, and make it fit for thyself in
//!   the field; and afterwards build thine house."*
//!   Precise coordinate tracking and stream traversal prepare clean raw glyphs before
//!   syntactic assembly.
//! - **Proverbs 22:28**: *"Remove not the ancient landmark, which thy fathers have set."*
//!   Preserves reel positions, line markers, and boundary checkpoints without loss.
//!
//! ## Universal Cursor Invariants:
//! 1. **1-Indexed Coordinates**: Lines and columns begin at 1 for human diagnostic clarity.
//! 2. **CRLF Normalization**: Both `\r\n` and `\n` uniformly advance line coordinates by exactly 1.
//! 3. **Zero-Heap Matching**: Compares string literals directly against character iterators.
//! 4. **Speculative Rewind**: Checkpointing allows lossless rollback for ambiguous prefixes.

use crate::parser::tokens::SourceLocation;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Checkpoint State & Character Cursor Structure )
// ============================================================================

/// Immutable snapshot of cursor state for speculative lookahead and backtracking.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct CharCheckpoint {
    /// Character index position in reel.
    pub pos: usize,
    /// 1-indexed line coordinate.
    pub line: usize,
    /// 1-indexed column coordinate.
    pub col: usize,
    /// Byte offset within original source string.
    pub offset: usize,
}

/// Universal cursor over a sequence of Unicode characters with coordinate tracking.
#[derive(Debug, Clone)]
pub struct CharCursor<'a> {
    /// Borrowed raw source reel string.
    source: &'a str,
    /// Decoded Unicode characters representing the source reel tape.
    chars: Vec<char>,
    /// Current character index.
    pos: usize,
    /// Active line number (1-indexed).
    line: usize,
    /// Active column number (1-indexed).
    col: usize,
    /// Active byte offset within raw source text.
    offset: usize,
}

impl<'a> CharCursor<'a> {
    /// Constructs a new [`CharCursor`] over the provided UTF-8 source reel string.
    pub fn new(source: &'a str) -> Self {
        Self {
            source,
            chars: source.chars().collect(),
            pos: 0,
```

```
            line: 1,
            col: 1,
            offset: 0,
        }
    }

    /// Returns the borrowed raw source reel string.
    #[inline]
    pub fn source(&self) -> &'a str {
        self.source
    }

    /// Current 1-indexed line number.
    #[inline]
    pub fn line(&self) -> usize {
        self.line
    }

    /// Current 1-indexed column number.
    #[inline]
    pub fn col(&self) -> usize {
        self.col
    }

    /// Current byte offset from start of source reel.
    #[inline]
    pub fn offset(&self) -> usize {
        self.offset
    }

    /// Captures the current cursor location as a [`SourceLocation`].
    #[inline]
    pub fn location(&self) -> SourceLocation {
        SourceLocation::new(self.line, self.col, self.offset)
    }

    /// Captures a lightweight checkpoint for backtracking.
    #[inline]
    pub fn checkpoint(&self) -> CharCheckpoint {
        CharCheckpoint {
            pos: self.pos,
            line: self.line,
            col: self.col,
            offset: self.offset,
        }
    }

    /// Restores the cursor state to a previously captured checkpoint.
    #[inline]
    pub fn restore(&mut self, cp: CharCheckpoint) {
        self.pos = cp.pos;
        self.line = cp.line;
        self.col = cp.col;
        self.offset = cp.offset;
    }

    /// Returns `true` if the cursor has reached or surpassed the end of the character reel.
    #[inline]
    pub fn is_at_end(&self) -> bool {
        self.pos >= self.chars.len()
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Universal Character Navigation, Lookahead & Scanning Primitives )
// ============================================================================
impl<'a> CharCursor<'a> {
    /// Inspects the current character without advancing.
    #[inline]
    pub fn peek(&self) -> Option<char> {
        self.peek_at(0)
    }

    /// Inspects a character at `offset` positions ahead of current cursor without advancing.
    #[inline]
    pub fn peek_at(&self, offset: usize) -> Option<char> {
```

```
        let idx = self.pos + offset;
        if idx < self.chars.len() {
            Some(self.chars[idx])
        } else {
            None
        }
    }

    /// Tests whether the character at `offset` satisfies the given predicate.
    #[inline]
    pub fn peek_matches<F: Fn(char) -> bool>(&self, offset: usize, predicate: F) -> bool {
        self.peek_at(offset).map_or(false, predicate)
    }

    /// Finds the next non-whitespace character starting from `start_offset` ahead.
    #[inline]
    pub fn peek_next_non_whitespace(&self, start_offset: usize) -> Option<char> {
        let mut off = start_offset;
        while let Some(ch) = self.peek_at(off) {
            if !ch.is_whitespace() {
                return Some(ch);
            }
            off += 1;
        }
        None
    }

    /// Advances the cursor by one character, updating line, column, and byte offset.
    ///
    /// Uniformly normalizes `\r\n` (CRLF) and `\n` to prevent spurious line jumps.
    pub fn advance(&mut self) -> Option<char> {
        if self.pos >= self.chars.len() {
            return None;
        }

        let ch = self.chars[self.pos];
        self.pos += 1;
        self.offset += ch.len_utf8();

        if ch == '\r' {
            // Check for CRLF sequence
            if self.pos < self.chars.len() && self.chars[self.pos] == '\n' {
                self.pos += 1;
                self.offset += 1; // '\n' is 1 byte
            }
            self.line += 1;
            self.col = 1;
        } else if ch == '\n' {
            self.line += 1;
            self.col = 1;
        } else {
            self.col += 1;
        }

        Some(ch)
    }

    /// Conditionally consumes the current character if it matches `expected`.
    #[inline]
    pub fn match_char(&mut self, expected: char) -> bool {
        if self.peek() == Some(expected) {
            self.advance();
            true
        } else {
            false
        }
    }

    /// Matches and consumes the exact target string at cursor without heap allocations.
    pub fn match_str(&mut self, target: &str) -> bool {
        let target_len = target.chars().count();
        if self.pos + target_len > self.chars.len() {
            return false;
        }

        for (i, target_ch) in target.chars().enumerate() {
            if self.chars[self.pos + i] != target_ch {
                return false;
```

```
            }
        }

        for _ in 0..target_len {
            self.advance();
        }
        true
    }

    /// Repeatedly consumes characters while `predicate` returns `true`, accumulating them.
    pub fn consume_while<F: Fn(char) -> bool>(&mut self, predicate: F) -> String {
        let mut result = String::new();
        while let Some(ch) = self.peek() {
            if predicate(ch) {
                result.push(ch);
                self.advance();
            } else {
                break;
            }
        }
        result
    }

    /// Consumes contiguous whitespace characters (spaces, tabs, newlines).
    pub fn skip_whitespace(&mut self) {
        while let Some(ch) = self.peek() {
            if ch.is_whitespace() {
                self.advance();
            } else {
                break;
            }
        }
    }

    /// Consumes a single-line comment initiated by `comment_char` up to the next newline.
    pub fn skip_line_comment(&mut self, comment_char: char) -> bool {
        if self.peek() == Some(comment_char) {
            while let Some(ch) = self.peek() {
                if ch == '\n' || ch == '\r' {
                    break;
                }
                self.advance();
            }
            true
        } else {
            false
        }
    }

    /// Universally consumes whitespace and comments starting with `comment_char`.
    pub fn skip_whitespace_and_comments(&mut self, comment_char: char) {
        loop {
            self.skip_whitespace();
            if !self.skip_line_comment(comment_char) {
                break;
            }
        }
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Universal Character Stream Verification Proofs & Traversal Oracles )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_stream_linear_navigation_and_crlf() {
        let mut cursor = CharCursor::new("A\r\nB\nC");
        assert_eq!(cursor.line(), 1);
        assert_eq!(cursor.col(), 1);

        assert_eq!(cursor.advance(), Some('A'));
        assert_eq!(cursor.line(), 1);
        assert_eq!(cursor.col(), 2);
```

```
308
309         // CRLF normalized advance
310         assert_eq!(cursor.advance(), Some('\r'));
311         assert_eq!(cursor.line(), 2);
312         assert_eq!(cursor.col(), 1);
313
314         assert_eq!(cursor.advance(), Some('B'));
315         assert_eq!(cursor.advance(), Some('\n'));
316         assert_eq!(cursor.line(), 3);
317         assert_eq!(cursor.col(), 1);
318
319         assert_eq!(cursor.advance(), Some('C'));
320         assert_eq!(cursor.advance(), None);
321         assert!(cursor.is_at_end());
322     }
323
324     #[test]
325     fn test_zero_allocation_match_str() {
326         let mut cursor = CharCursor::new("--> INSCRIBE");
327         assert!(cursor.match_str("-->"));
328         assert_eq!(cursor.col(), 4);
329
330         cursor.skip_whitespace();
331         assert!(cursor.match_str("INSCRIBE"));
332         assert!(cursor.is_at_end());
333     }
334
335     #[test]
336     fn test_checkpoint_and_backtracking() {
337         let mut cursor = CharCursor::new("Character` <= [ 1 ]");
338         let cp = cursor.checkpoint();
339
340         let ident = cursor.consume_while(|c| c.is_alphanumeric() || c == '`');
341         assert_eq!(ident, "Character`");
342
343         // Rollback to origin
344         cursor.restore(cp);
345         assert_eq!(cursor.line(), 1);
346         assert_eq!(cursor.col(), 1);
347         assert_eq!(cursor.peek(), Some('C'));
348     }
349
350     #[test]
351     fn test_whitespace_and_line_comments() {
352         let mut cursor = CharCursor::new("   # This is a comment\n42");
353         cursor.skip_whitespace_and_comments('#');
354         assert_eq!(cursor.peek(), Some('4'));
355         assert_eq!(cursor.line(), 2);
356     }
357 }
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/universal/cursor.rs RS 324 LINES 1248 WORDS 12.6 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal Token Cursor: Stream Navigation, Lookahead & Failpoint Guards )
// ================================================================================
// SCRIPTURE ANCHORS & EPISTEMIC WITNESSES:
//   - Exodus 31:18: "written with the finger of God" — Inscription of eternal truth.
//   - Proverbs 24:27: "Prepare thy work without, and make it fit for thyself in the field;
//                      and afterwards build thine house." — Systematic modularization.
//   - Proverbs 22:28: "Remove not the ancient landmark, which thy fathers have set."
//                      — Preservation of invariant boundaries and balanced truth.
//
// ARCHITECTURAL TOPOLOGY:
//   The TokenCursor isolates universal token stream traversal from grammar-specific
//   parsing rules. It models the lexical stream as a discrete, 1-dimensional causal
//   ribbon with deterministic cursor navigation, arbitrary lookahead, and failpoint
//   reporting via the diagnostic debugger.
// ================================================================================

use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::FailpointType;
use crate::parser::lexer::{Token, TokenType};

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Cursor State Initialization, Stream Context & Encapsulation )
// ================================================================================

/// Universal stream cursor managing position, lookahead, and delimiter integrity.
pub struct TokenCursor {
    /// Ingested token stream terminated by [`TokenType::Eof`].
    tokens: Vec<Token>,
    /// Current 0-based token index.
    pos: usize,
    /// Diagnostic debugger tracking boundary depth and failpoint telemetry.
    pub debugger: OmniDebugger,
}

impl TokenCursor {
    /// Constructs a new [`TokenCursor`] over a linear token sequence.
    ///
    /// # Arguments
    /// * `tokens` - Ingested token stream from the Lexer.
    /// * `debugger` - Diagnostic debugger enforcing failpoints and boundary invariants.
    pub fn new(tokens: Vec<Token>, debugger: OmniDebugger) -> Self {
        Self {
            tokens,
            pos: 0,
            debugger,
        }
    }

    /// Returns the current 0-based cursor position.
    #[inline]
    pub fn pos(&self) -> usize {
        self.pos
    }

    /// Explicitly sets the cursor position (used during backtracking or macro replay).
    #[inline]
    pub fn set_pos(&mut self, new_pos: usize) {
        self.pos = new_pos.min(self.tokens.len().saturating_sub(1));
    }

    /// Returns the total number of tokens in the stream.
    #[inline]
    pub fn len(&self) -> usize {
        self.tokens.len()
    }

    /// Returns true if the token stream contains no tokens.
    #[inline]
    pub fn is_empty(&self) -> bool {
        self.tokens.is_empty()
    }
```

```rust

    /// Returns the remaining token count from the current cursor position.
    #[inline]
    pub fn remaining(&self) -> usize {
        self.tokens.len().saturating_sub(self.pos)
    }

    /// Returns a slice of the underlying token stream.
    #[inline]
    pub fn tokens(&self) -> &[Token] {
        &self.tokens
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Universal Cursor Primitives: Inspection, Advancement, Matching & Expectation )
// ============================================================================

impl TokenCursor {
    // ------------------------------------------------------------------------
    // 1. UNIVERSAL INSPECTION (PURE LOOKAHEAD, ZERO SIDE-EFFECTS)
    // ------------------------------------------------------------------------

    /// Inspects the current token at the cursor without advancing.
    /// Returns the final EOF token if the cursor has exceeded the stream boundary.
    #[inline]
    pub fn peek(&self) -> &Token {
        if self.pos < self.tokens.len() {
            &self.tokens[self.pos]
        } else {
            self.tokens.last().expect("Token stream must have at least an EOF token")
        }
    }

    /// Inspects a future token at `offset` positions ahead of the cursor.
    #[inline]
    pub fn peek_at(&self, offset: usize) -> &Token {
        let target_idx = self.pos + offset;
        if target_idx < self.tokens.len() {
            &self.tokens[target_idx]
        } else {
            self.tokens.last().expect("Token stream must have at least an EOF token")
        }
    }

    /// Checks if the current token matches the expected token type without consuming it.
    #[inline]
    pub fn check(&self, expected: &TokenType) -> bool {
        &self.peek().token_type == expected
    }

    /// Checks if a lookahead token at `offset` matches the expected token type.
    #[inline]
    pub fn check_at(&self, offset: usize, expected: &TokenType) -> bool {
        &self.peek_at(offset).token_type == expected
    }

    /// Checks whether the cursor has reached the end-of-file token or stream termination.
    #[inline]
    pub fn is_at_end(&self) -> bool {
        self.check(&TokenType::Eof) || self.pos >= self.tokens.len()
    }

    /// Returns the (line, col) coordinates of the current token.
    #[inline]
    pub fn current_location(&self) -> (usize, usize) {
        let tok = self.peek();
        (tok.line, tok.col)
    }

    // ------------------------------------------------------------------------
    // 2. UNIVERSAL ADVANCEMENT & CONSUMPTION (STATE ADVANCEMENT)
    // ------------------------------------------------------------------------

    /// Advances the cursor by one token and returns the consumed token.
    #[inline]
    pub fn advance(&mut self) -> Token {
```

```
        let tok = self.peek().clone();
        if self.pos < self.tokens.len() {
            self.pos += 1;
        }
        tok
    }

    /// Conditionally consumes the current token if it matches `expected`.
    pub fn match_token(&mut self, expected: &TokenType) -> bool {
        if self.check(expected) {
            self.advance();
            true
        } else {
            false
        }
    }

    /// Consumes the current token if the predicate returns true.
    pub fn consume_if<F: FnOnce(&TokenType) -> bool>(&mut self, predicate: F) -> Option<Token> {
        if predicate(&self.peek().token_type) {
            Some(self.advance())
        } else {
            None
        }
    }

    // ---------------------------------------------------------------------------
    // 3. UNIVERSAL EXPECTATION & FAILPOINT GUARDS
    // ---------------------------------------------------------------------------

    /// Asserts that the current token matches `expected`, consuming it on match
    /// or emitting a structured syntax failpoint on mismatch.
    pub fn expect(&mut self, expected: &TokenType, context: &str) -> Result<Token, String> {
        let current = self.peek();
        if &current.token_type == expected {
            Ok(self.advance())
        } else {
            let msg = format!(
                "Expected token '{:?}' in {}, found '{:?}' ('{}') at L{}:C{}",
                expected, context, current.token_type, current.lexeme, current.line, current.col
            );
            self.debugger.logger.failpoint(FailpointType::SyntaxAnomaly, "CURSOR", &msg);
            Err(format!("FAILPOINT: {msg}"))
        }
    }

    /// Asserts that the current token is an identifier, consuming and extracting its name.
    pub fn expect_identifier(&mut self, context: &str) -> Result<(String, Token), String> {
        let current = self.peek().clone();
        match current.token_type {
            TokenType::Identifier(ref name) => {
                let id = name.clone();
                let tok = self.advance();
                Ok((id, tok))
            }
            TokenType::Who | TokenType::What | TokenType::When | TokenType::Where | TokenType::Why | TokenType::How => {
                let id = current.lexeme.clone();
                let tok = self.advance();
                Ok((id, tok))
            }
            _ => {
                let msg = format!(
                    "Expected identifier in {}, found '{:?}' ('{}') at L{}:C{}",
                    context, current.token_type, current.lexeme, current.line, current.col
                );
                self.debugger.logger.failpoint(FailpointType::SyntaxAnomaly, "CURSOR", &msg);
                Err(format!("FAILPOINT: {msg}"))
            }
        }
    }

    /// Asserts that the current token opens a morphological boundary (`[`, `{`, `<`),
    /// recording it into the active boundary stack.
    pub fn expect_boundary_open(&mut self, context: &str) -> Result<(char, Token), String> {
        let tok = self.peek().clone();
        let open_char = match tok.token_type {
            TokenType::LBracket => '[',
            TokenType::LBrace => '{',
```

```
            TokenType::LAngle => '<',
            _ => {
                let msg = format!(
                    "Expected boundary opening delimiter ('[', '{{', '<') in {}, found '{:?}' at L{}:C{}",
                    context, tok.token_type, tok.line, tok.col
                );
                self.debugger.logger.failpoint(FailpointType::SyntaxAnomaly, "CURSOR", &msg);
                return Err(format!("FAILPOINT: {msg}"));
            }
        };

        self.advance();
        self.debugger.push_boundary(open_char, tok.line, tok.col);
        Ok((open_char, tok))
    }

    /// Asserts that the current token closes a morphological boundary (`]`, `}`, `>`),
    /// validating and popping it from the active boundary stack.
    pub fn expect_boundary_close(&mut self, context: &str) -> Result<(crate::core::boundaries::BoundaryType, Token),
String> {
        if self.is_at_end() {
            let msg = format!("Unclosed boundary in {} reaches EOF", context);
            self.debugger.logger.failpoint(FailpointType::BoundaryLeak, "CURSOR", &msg);
            return Err(format!("FAILPOINT: {msg}"));
        }

        let tok = self.peek().clone();
        let close_char = match tok.token_type {
            TokenType::RBracket => ']',
            TokenType::RBrace => '}',
            TokenType::RAngle => '>',
            _ => {
                let msg = format!(
                    "Expected boundary closing delimiter (']', '}}', '>') in {}, found '{:?}' at L{}:C{}",
                    context, tok.token_type, tok.line, tok.col
                );
                self.debugger.logger.failpoint(FailpointType::SyntaxAnomaly, "CURSOR", &msg);
                return Err(format!("FAILPOINT: {msg}"));
            }
        };

        self.advance();
        let btype = self.debugger.pop_boundary(close_char, tok.line, tok.col)?;
        Ok((btype, tok))
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Cursor Invariant Verification, Boundary Mechanics & Unit Proofs )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;
    use crate::diagnostics::logger::DiagnosticLogger;

    fn create_test_cursor(src: &str) -> TokenCursor {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 10);
        let mut lexer = crate::parser::lexer::Lexer::new(src, logger);
        let tokens = lexer.tokenize().expect("Lexing must succeed");
        TokenCursor::new(tokens, debugger)
    }

    #[test]
    fn test_cursor_linear_advance_and_peek() {
        let mut cursor = create_test_cursor("[ Roll ] --> 42");
        assert_eq!(cursor.peek().token_type, TokenType::LBracket);
        assert_eq!(cursor.peek_at(1).lexeme, "Roll");
        assert_eq!(cursor.advance().token_type, TokenType::LBracket);
        assert_eq!(cursor.peek().lexeme, "Roll");
        assert!(cursor.match_token(&TokenType::Identifier("Roll".into())));
        assert_eq!(cursor.peek().token_type, TokenType::RBracket);
    }

    #[test]
    fn test_cursor_boundary_open_and_close_physics() {
```

```
        let mut cursor = create_test_cursor("{ Character ]");
        let (open_char, _) = cursor.expect_boundary_open("TestBoundary").expect("Open boundary");
        assert_eq!(open_char, '{');
        assert_eq!(cursor.advance().lexeme, "Character");
        let (btype, _) = cursor.expect_boundary_close("TestBoundary").expect("Close boundary");
        assert_eq!(btype, crate::core::boundaries::BoundaryType::TopDown);
    }

    #[test]
    fn test_cursor_safe_lookahead_past_eof() {
        let cursor = create_test_cursor("Alpha");
        assert_eq!(cursor.peek().lexeme, "Alpha");
        assert_eq!(cursor.peek_at(1).token_type, TokenType::Eof);
        assert_eq!(cursor.peek_at(100).token_type, TokenType::Eof);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/universal/mod.rs RS 20 LINES 95 WORDS 0.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal Stream Navigation Subsystem Identity & Exports )
// ============================================================================
//! # Universal Stream & Navigation Machinery
//!
//! Provides pure, language-agnostic character-reel and token-stream cursors with
//! arbitrary lookahead, coordinate tracking, and speculative checkpointing.
//!
//! ## Scriptural Grounding:
//! - **Exodus 31:18**: Inscription of truth on physical reels.
//! - **Proverbs 24:27**: Preparing the raw stream in the field before assembly.
//! - **Proverbs 22:28**: Preserving stream landmarks and boundary coordinates.

pub mod char_stream;
pub mod cursor;

pub use char_stream::{CharCheckpoint, CharCursor};
pub use cursor::TokenCursor;
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/syntactic/grammar.rs RS 599 LINES 1985 WORDS 24.3 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniCode Morphological Grammar: Precedence Cascade & Dual-Vector Physics )
// ============================================================================
// SCRIPTURE ANCHORS & EPISTEMIC WITNESSES:
//   - Exodus 31:18: "written with the finger of God" — Inscription of eternal truth.
//   - Proverbs 24:27: "Prepare thy work without, and make it fit for thyself in the field;
//                      and afterwards build thine house." — Systematic modularization.
//   - Proverbs 22:28: "Remove not the ancient landmark, which thy fathers have set."
//                      — Preservation of invariant boundaries and balanced truth.
//
// DUAL-VECTOR ARCHITECTURE:
//   1. FLOW DOWN (Decomposition / Macrocosm to Microcosm):
//      Program -> Statement -> Caveat/Connective (L1) -> Directional (L2) ->
//      Relation (L3) -> Arithmetic (L4) -> Unary (L5) -> Primary (L6).
//
//   2. FLOW BACK UP (Synthesis / Inscription & Folding):
//      Primary atoms fold upward into binary relations, directional flows,
//      and first-class intent asides `( ... )`, anchoring into complete rolls.
// ============================================================================

use crate::ast::nodes::*;
use crate::core::operators::{BeingRelation, Connective, DirectionalVector};
use crate::parser::cursor::TokenCursor;
use crate::parser::lexer::TokenType;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Grammar Engine Context & State Mounting )
// ============================================================================

/// Grammar engine driving recursive-descent parsing over a [`TokenCursor`].
pub struct GrammarEngine<'a> {
    /// Active token stream cursor providing lookahead and expectation guards.
    pub cursor: &'a mut TokenCursor,
}

impl<'a> GrammarEngine<'a> {
    /// Constructs a new [`GrammarEngine`] mounted over a [`TokenCursor`].
    pub fn new(cursor: &'a mut TokenCursor) -> Self {
        Self { cursor }
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Morphological Grammars: Statements, Boundaries, Downward & Upward Cascades )
// ============================================================================

impl<'a> GrammarEngine<'a> {
    // ------------------------------------------------------------------------
    // PART 3A: MACROCOSMIC PROGRAM & STATEMENT PARSING
    // ------------------------------------------------------------------------

    /// Parses an entire source reel or tape into a unified [`ProgramNode`].
    ///
    /// Iterates through sequential top-level statements until EOF, then proves
    /// that all opened boundary frames have been cleanly popped from the stack.
    pub fn parse_program(&mut self) -> Result<ProgramNode, String> {
        let mut statements = Vec::new();

        while !self.cursor.is_at_end() {
            let stmt = self.parse_statement()?;
            statements.push(stmt);
        }

        self.cursor.debugger.assert_all_boundaries_closed()?;
        Ok(ProgramNode { statements, line: 1, col: 1 })
    }

    /// Parses a single statement or top-level structural declaration.
    ///
    /// Recognizes:
    /// 1. Root assignment declarations: `= [Module]: ...` and `= [Tape]: ...`
```

```
    /// 2. Section blocks: `Opening { ... }`, `Body [ ... ]`, `Closing ( Aside ) { ... }`
    /// 3. Attached body blocks: `< Character > = ... ( Aside ) { ... }`
    /// 4. Expressions, entities, and directional transitions.
    pub fn parse_statement(&mut self) -> Result<ASTNode, String> {
        // 1. Handle optional leading '=' for root module or tape declarations:
        if self.cursor.check(&TokenType::Is) {
            let eq_tok = self.cursor.advance();
            let expr = self.parse_expression()?;

            // Check if immediately followed by an attached body boundary block
            if matches!(self.cursor.peek().token_type, TokenType::LBrace | TokenType::LBracket) {
                let body_boundary = self.parse_boundary()?;
                return Ok(ASTNode::Connective(ConnectiveNode {
                    left: Box::new(expr),
                    connective: Connective::InRelationTo,
                    right: Box::new(ASTNode::Boundary(body_boundary)),
                    line: eq_tok.line,
                    col: eq_tok.col,
                }));
            }

            return Ok(expr);
        }

        // 2. Parse base expression
        let expr = self.parse_expression()?;

        // 3. Flow Back Up: Attach optional trailing Aside and Contextual/Denotational block
        let maybe_aside = if let TokenType::Aside(ref text) = self.cursor.peek().token_type {
            Some(text.clone())
        } else {
            None
        };

        if let Some(aside_text) = maybe_aside {
            if matches!(self.cursor.peek_at(1).token_type, TokenType::LBrace | TokenType::LBracket) {
                let a_tok = self.cursor.advance(); // consume Aside
                let mut body_block = self.parse_boundary()?;
                if body_block.aside.is_none() {
                    body_block.aside = Some(AsideNode {
                        intent: aside_text,
                        line: a_tok.line,
                        col: a_tok.col,
                    });
                }
                return Ok(ASTNode::Connective(ConnectiveNode {
                    left: Box::new(expr),
                    connective: Connective::InRelationTo,
                    right: Box::new(ASTNode::Boundary(body_block)),
                    line: a_tok.line,
                    col: a_tok.col,
                }));
            }
        }

        // Check if directly followed by an attached body boundary block without aside
        if matches!(self.cursor.peek().token_type, TokenType::LBrace | TokenType::LBracket) {
            let body_block = self.parse_boundary()?;
            return Ok(ASTNode::Connective(ConnectiveNode {
                left: Box::new(expr),
                connective: Connective::InRelationTo,
                right: Box::new(ASTNode::Boundary(body_block)),
                line: self.cursor.peek().line,
                col: self.cursor.peek().col,
            }));
        }

        Ok(expr)
    }

    // ---------------------------------------------------------------------------
    // PART 3B: MORPHOLOGICAL BOUNDARY ENGINE (THE 6 BOUNDARY RANKS)
    // ---------------------------------------------------------------------------

    /// Parses a morphological boundary block, enforcing boundary stack physics:
    ///   - Rank 1: Invariant Axiom       `< ... >`
    ///   - Rank 2: Denotation            `[ ... ]`
    ///   - Rank 3: Connotation           `{ ... }`
```

```
    ///   - Rank 4: Inside-Out Emergence  `[ ... }`
    ///   - Rank 5: Outside-In Imposition `{ ... ]`
    ///   - Rank 6: Intent Aside          `( ... )`
    pub fn parse_boundary(&mut self) -> Result<BoundaryNode, String> {
        let (_open_char, open_tok) = self.cursor.expect_boundary_open("BoundaryBlock")?;
        let mut elements = Vec::new();

        while !self.cursor.is_at_end() && !matches!(
            self.cursor.peek().token_type,
            TokenType::RBracket | TokenType::RBrace | TokenType::RAngle
        ) {
            let el = self.parse_expression()?;
            elements.push(el);
        }

        let (btype, _close_tok) = self.cursor.expect_boundary_close("BoundaryBlock")?;

        // Flow Back Up: Check for attached first-class Aside: ( ... )
        let mut aside = None;
        let maybe_aside = if let TokenType::Aside(ref text) = self.cursor.peek().token_type {
            Some(text.clone())
        } else {
            None
        };
        if let Some(text) = maybe_aside {
            let a_tok = self.cursor.advance();
            aside = Some(AsideNode {
                intent: text,
                line: a_tok.line,
                col: a_tok.col,
            });
        }

        Ok(BoundaryNode {
            boundary_type: btype,
            elements,
            aside,
            line: open_tok.line,
            col: open_tok.col,
        })
    }

    // ---------------------------------------------------------------------------
    // PART 3C: DOWNWARD PRECEDENCE CASCADE (FLOW DOWN)
    // ---------------------------------------------------------------------------

    /// Entry point for expression hierarchy parsing.
    #[inline]
    pub fn parse_expression(&mut self) -> Result<ASTNode, String> {
        self.parse_caveat_or_connective()
    }

    /// Precedence Level 1: Adversative Caveat `!` (BUT / Guard Interlock)
    /// and Connectives `::` / `&` (And), `|` (Or), `:` / `@` / `::@` (In Relation To / Located At).
    pub fn parse_caveat_or_connective(&mut self) -> Result<ASTNode, String> {
        let mut expr = self.parse_directional()?;

        while matches!(
            self.cursor.peek().token_type,
            TokenType::But | TokenType::And | TokenType::HardwareLocator | TokenType::Or | TokenType::InRelationTo |
TokenType::BoundPair | TokenType::At
        ) {
            let op_tok = self.cursor.advance();
            let connective = match op_tok.token_type {
                TokenType::But => Connective::But,
                TokenType::And | TokenType::BoundPair | TokenType::HardwareLocator => Connective::And,
                TokenType::Or => Connective::Or,
                TokenType::InRelationTo | TokenType::At => Connective::InRelationTo,
                _ => unreachable!(),
            };
            let right = self.parse_directional()?;
            expr = ASTNode::Connective(ConnectiveNode {
                left: Box::new(expr),
                connective,
                right: Box::new(right),
                line: op_tok.line,
                col: op_tok.col,
            });
```

```
        }

        Ok(expr)
    }

    /// Precedence Level 2: Directional Vectors across X, Y, and Z axes:
    ///   `-->` (ActUpon), `<--` (ActAgain), `^--` (BuildsTo), `v--` (BreaksDownFrom),
    ///   as well as scale modulations and quadrant vectors.
    pub fn parse_directional(&mut self) -> Result<ASTNode, String> {
        let mut expr = self.parse_relation()?;

        while matches!(
            self.cursor.peek().token_type,
            TokenType::ActUpon
                | TokenType::ActAgain
                | TokenType::BuildsTo
                | TokenType::BreaksDownFrom
                | TokenType::Major
                | TokenType::Minor
                | TokenType::Great
                | TokenType::Small
                | TokenType::UpperRight
                | TokenType::LowerRight
                | TokenType::UpperLeft
                | TokenType::LowerLeft
        ) {
            let op_tok = self.cursor.advance();
            let vector = match op_tok.token_type {
                TokenType::ActUpon => DirectionalVector::ActUpon,
                TokenType::ActAgain => DirectionalVector::ActAgain,
                TokenType::BuildsTo => DirectionalVector::BuildsTo,
                TokenType::BreaksDownFrom => DirectionalVector::BreaksDownFrom,
                TokenType::Major => DirectionalVector::Major,
                TokenType::Minor => DirectionalVector::Minor,
                TokenType::Great => DirectionalVector::Great,
                TokenType::Small => DirectionalVector::Small,
                TokenType::UpperRight => DirectionalVector::UpperRight,
                TokenType::LowerRight => DirectionalVector::LowerRight,
                TokenType::UpperLeft => DirectionalVector::UpperLeft,
                TokenType::LowerLeft => DirectionalVector::LowerLeft,
                _ => unreachable!(),
            };
            let target = self.parse_relation()?;
            expr = ASTNode::Directional(DirectionalNode {
                source: Box::new(expr),
                vector,
                target: Box::new(target),
                line: op_tok.line,
                col: op_tok.col,
            });
        }

        Ok(expr)
    }

    /// Precedence Level 3: Being Relations and Invariant Equivalence:
    ///   `=` (Is), `!=` (IsNot), `~=` (ToBeLike), `<=>` (InvariantEquivalence / Transduction),
    ///   `<`, `>`, `<=`, `>=` (Relational Bound Comparisons).
    pub fn parse_relation(&mut self) -> Result<ASTNode, String> {
        let mut expr = self.parse_arithmetic()?;

        while matches!(
            self.cursor.peek().token_type,
            TokenType::Is
                | TokenType::IsNot
                | TokenType::ToBeNot
                | TokenType::ToBeLike
                | TokenType::InvariantEquivalence
                | TokenType::LessThan
                | TokenType::GreaterThan
                | TokenType::LessEqual
                | TokenType::GreaterEqual
        ) {
            let op_tok = self.cursor.advance();
            let relation = match op_tok.token_type {
                TokenType::Is => BeingRelation::Is,
                TokenType::IsNot => BeingRelation::IsNot,
                TokenType::ToBeNot => BeingRelation::ToBeNot,
```

```
                TokenType::ToBeLike => BeingRelation::ToBeLike,
                TokenType::InvariantEquivalence => BeingRelation::Equivalence,
                TokenType::LessThan => BeingRelation::LessThan,
                TokenType::GreaterThan => BeingRelation::GreaterThan,
                TokenType::LessEqual => BeingRelation::LessEqual,
                TokenType::GreaterEqual => BeingRelation::GreaterEqual,
                _ => unreachable!(),
            };
            let right = self.parse_arithmetic()?;
            expr = ASTNode::Relation(RelationNode {
                left: Box::new(expr),
                relation,
                right: Box::new(right),
                line: op_tok.line,
                col: op_tok.col,
            });
        }

        Ok(expr)
    }

    /// Precedence Level 4: SVO Arithmetic Math Keywords:
    ///   `plus`, `minus`, `times`, `div`.
    pub fn parse_arithmetic(&mut self) -> Result<ASTNode, String> {
        let mut expr = self.parse_unary()?;

        while matches!(
            self.cursor.peek().token_type,
            TokenType::Plus | TokenType::Minus | TokenType::Times | TokenType::Div
        ) {
            let op_tok = self.cursor.advance();
            let op = match op_tok.token_type {
                TokenType::Plus => ArithmeticOp::Add,
                TokenType::Minus => ArithmeticOp::Sub,
                TokenType::Times => ArithmeticOp::Mul,
                TokenType::Div => ArithmeticOp::Div,
                _ => unreachable!(),
            };
            let right = self.parse_unary()?;
            expr = ASTNode::Arithmetic(ArithmeticNode {
                left: Box::new(expr),
                op,
                right: Box::new(right),
                line: op_tok.line,
                col: op_tok.col,
            });
        }

        Ok(expr)
    }

    /// Precedence Level 5: Unary Operations:
    ///   `--` (Not) and Prefix Vectors (`-->`, `<--`, `^--`, `v--`).
    pub fn parse_unary(&mut self) -> Result<ASTNode, String> {
        if self.cursor.check(&TokenType::Not) {
            let op_tok = self.cursor.advance();
            let operand = self.parse_unary()?;
            return Ok(ASTNode::UnaryNot(UnaryNotNode {
                operand: Box::new(operand),
                line: op_tok.line,
                col: op_tok.col,
            }));
        }

        if matches!(
            self.cursor.peek().token_type,
            TokenType::ActAgain | TokenType::ActUpon | TokenType::BuildsTo | TokenType::BreaksDownFrom
        ) {
            let op_tok = self.cursor.advance();
            let vector = match op_tok.token_type {
                TokenType::ActAgain => DirectionalVector::ActAgain,
                TokenType::ActUpon => DirectionalVector::ActUpon,
                TokenType::BuildsTo => DirectionalVector::BuildsTo,
                TokenType::BreaksDownFrom => DirectionalVector::BreaksDownFrom,
                _ => unreachable!(),
            };
            let operand = self.parse_unary()?;
            return Ok(ASTNode::Directional(DirectionalNode {
```

```
                source: Box::new(ASTNode::Ternary(crate::core::ternary::QuadState::Proceed)),
                vector,
                target: Box::new(operand),
                line: op_tok.line,
                col: op_tok.col,
            }));
        }

        self.parse_primary()
    }

    // ------------------------------------------------------------------------
    // PART 3D: UPWARD INSCRIPTION & SYNTHESIS (FLOW BACK UP)
    // ------------------------------------------------------------------------

    /// Precedence Level 6: Primaries & Atom Synthesis:
    ///   Morphological Boundaries, Affixed Polarities, Boundary Casts, Asides,
    ///   Ternary Literals, Numbers, Strings, Identifiers (with state-primes `),
    ///   and Complex Entities with Definitions, Asides, and Contexts.
    pub fn parse_primary(&mut self) -> Result<ASTNode, String> {
        let tok = self.cursor.peek().clone();
        match tok.token_type {
            TokenType::LBracket | TokenType::LBrace | TokenType::LAngle => {
                let b = self.parse_boundary()?;
                Ok(ASTNode::Boundary(b))
            }
            TokenType::AffixedIdent { ref name, polarity, position } => {
                let id_name = name.clone();
                let line = tok.line;
                let col = tok.col;
                self.cursor.advance();
                Ok(ASTNode::Polarity(PolarityNode {
                    name: id_name,
                    polarity,
                    position,
                    line,
                    col,
                }))
            }
            TokenType::BoundaryCast { boundary, polarity } => {
                let line = tok.line;
                let col = tok.col;
                self.cursor.advance();
                Ok(ASTNode::BoundaryCast(BoundaryCastNode {
                    boundary,
                    polarity,
                    line,
                    col,
                }))
            }
            TokenType::Aside(ref intent) => {
                self.cursor.advance();
                Ok(ASTNode::Aside(AsideNode {
                    intent: intent.clone(),
                    line: tok.line,
                    col: tok.col,
                }))
            }
            TokenType::Ternary(state) => {
                self.cursor.advance();
                Ok(ASTNode::Ternary(state))
            }
            TokenType::Number(val) => {
                self.cursor.advance();
                Ok(ASTNode::Number(val))
            }
            TokenType::StringLit(ref s) => {
                self.cursor.advance();
                Ok(ASTNode::StringLit(s.clone()))
            }
            TokenType::CharLit(c) => {
                let s = c.to_string();
                self.cursor.advance();
                Ok(ASTNode::StringLit(s))
            }
            TokenType::Infer => {
                self.cursor.advance();
                Ok(ASTNode::Identifier("?".into()))
```

```
            }
            TokenType::Pronoun(ref p) => {
                let name = p.as_str().to_string();
                self.cursor.advance();
                Ok(ASTNode::Identifier(name))
            }
            TokenType::Identifier(_)
            | TokenType::Who
            | TokenType::What
            | TokenType::When
            | TokenType::Where
            | TokenType::Why
            | TokenType::How => {
                let id_str = match tok.token_type {
                    TokenType::Identifier(ref n) => n.clone(),
                    _ => tok.lexeme.clone(),
                };
                let next_tok = self.cursor.peek_at(1).token_type.clone();

                // 1. Entity Syntax: Name [ Definition ] ( Aside ) { Context }
                if matches!(next_tok, TokenType::LBracket | TokenType::LAngle | TokenType::LBrace) {
                    let id_name = id_str;
                    let line = tok.line;
                    let col = tok.col;
                    self.cursor.advance(); // consume identifier

                    let def_node = self.parse_boundary()?;
                    let mut aside_node = None;
                    let mut context_node = None;

                    // Check for optional Aside: ( ... )
                    let maybe_aside = if let TokenType::Aside(ref text) = self.cursor.peek().token_type {
                        Some(text.clone())
                    } else {
                        None
                    };
                    if let Some(text) = maybe_aside {
                        let a_tok = self.cursor.advance();
                        aside_node = Some(AsideNode {
                            intent: text,
                            line: a_tok.line,
                            col: a_tok.col,
                        });
                    }

                    // Check for optional Context boundary
                    if matches!(self.cursor.peek().token_type, TokenType::LBrace | TokenType::LBracket) {
                        context_node = Some(self.parse_boundary()?);
                    }

                    if def_node.boundary_type.is_immutable() {
                        self.cursor.debugger.register_immutable(&id_name);
                    }

                    return Ok(ASTNode::Entity(EntityNode {
                        name: id_name,
                        definition: def_node,
                        aside: aside_node,
                        context: context_node,
                        line,
                        col,
                    }));
                }

                // 2. Entity Syntax: Name ( Aside ) { Context }
                if let TokenType::Aside(ref text) = next_tok {
                    if matches!(self.cursor.peek_at(2).token_type, TokenType::LBrace | TokenType::LBracket |
TokenType::LAngle) {
                        let id_name = id_str;
                        let line = tok.line;
                        let col = tok.col;
                        let aside_text = text.clone();
                        self.cursor.advance(); // consume identifier
                        let a_tok = self.cursor.advance(); // consume aside
                        let def_node = self.parse_boundary()?;

                        return Ok(ASTNode::Entity(EntityNode {
                            name: id_name,
```

```
                            definition: def_node,
                            aside: Some(AsideNode {
                                intent: aside_text,
                                line: a_tok.line,
                                col: a_tok.col,
                            }),
                            context: None,
                            line,
                            col,
                        }));
                    }
                }

                self.cursor.advance();
                Ok(ASTNode::Identifier(id_str))
            }
            _ => {
                let msg = format!("Unexpected token '{:?}' in expression at L{}:C{}", tok.token_type, tok.line, tok.col);
                self.cursor.debugger.logger.failpoint(crate::diagnostics::logger::FailpointType::SyntaxAnomaly,
"GRAMMAR", &msg);
                Err(format!("FAILPOINT: {msg}"))
            }
        }
    }
}

// ==============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Grammar Verification, Boundary Integrity Proofs & Unit Tests )
// ==============================================================================

#[cfg(test)]
mod tests {
    use super::*;
    use crate::diagnostics::debugger::OmniDebugger;
    use crate::diagnostics::logger::DiagnosticLogger;

    fn parse_src(src: &str) -> Result<ProgramNode, String> {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 10);
        let mut lexer = crate::parser::lexer::Lexer::new(src, logger);
        let tokens = lexer.tokenize().expect("Lexing must succeed");
        let mut cursor = TokenCursor::new(tokens, debugger);
        let mut engine = GrammarEngine::new(&mut cursor);
        engine.parse_program()
    }

    #[test]
    fn test_grammar_precedence_downward_and_upward() {
        let prog = parse_src("1 plus 2 times 3 <=> 7").expect("Parse expression");
        assert_eq!(prog.statements.len(), 1);
    }

    #[test]
    fn test_grammar_directional_and_connective() {
        let prog = parse_src("{ Inflow ] --> INSCRIBE [ Cell ] = [ Character` ]").expect("Parse inscribe roll");
        assert_eq!(prog.statements.len(), 1);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/syntactic/mod.rs RS 20 LINES 93 WORDS 0.8 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Syntactic Grammar & Parser Subsystem )
// ================================================================================
//! # Syntactic Grammar & Parser Subsystem
//!
//! Builds the Abstract Syntax Tree (AST) by enforcing the 6 ranked morphological
//! boundaries and the 6-level dual-vector precedence cascade.
//!
//! ## Scriptural Grounding:
//! - **Exodus 31:18**: Building upon the inscribed tables of stone.
//! - **Proverbs 24:27**: Building the architectural house upon prepared tokens.
//! - **Proverbs 22:28**: Enforcing unalterable morphological boundaries.

pub mod grammar;
pub mod parser;

pub use grammar::GrammarEngine;
pub use parser::Parser;
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/syntactic/parser.rs RS 203 LINES 935 WORDS 8.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniCode Parser Coordinator: Grammar Engine & Token Stream Facade )
// ============================================================================
// SCRIPTURE ANCHORS & EPISTEMIC WITNESSES:
//   - Exodus 31:18: "written with the finger of God" — Inscription of eternal truth.
//   - Proverbs 24:27: "Prepare thy work without, and make it fit for thyself in the field;
//                      and afterwards build thine house." — Systematic modularization.
//   - Proverbs 22:28: "Remove not the ancient landmark, which thy fathers have set."
//                      — Preservation of invariant boundaries and balanced truth.
//
// ARCHITECTURAL DESIGN:
//   The Parser acts as the public coordinator unifying:
//     1. Universal TokenCursor (Stream navigation, lookahead & expectation guards).
//     2. Morphological GrammarEngine (Boundary physics & dual-vector precedence cascade).
//     3. Diagnostic Debugger (Boundary frame tracking & failpoint telemetry).
// ============================================================================

use crate::ast::nodes::*;
use crate::diagnostics::debugger::OmniDebugger;
use crate::parser::cursor::TokenCursor;
use crate::parser::grammar::GrammarEngine;
use crate::parser::lexer::{Token, TokenType};

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Parser State Initialization, Token Stream Mounting & Stream Delegations )
// ============================================================================

/// State container coordinating recursive-descent parsing across modular engines.
pub struct Parser {
    /// Universal stream cursor managing position, lookahead, and delimiter integrity.
    pub cursor: TokenCursor,
    /// Active debugger reflecting boundary stack physics and diagnostic failpoints.
    pub debugger: OmniDebugger,
}

impl Parser {
    /// Constructs a new [`Parser`] over an ingested token stream.
    ///
    /// # Arguments
    /// * `tokens` - Complete token stream terminated by [`TokenType::Eof`].
    /// * `debugger` - Diagnostic debugger enforcing delimiter integrity.
    pub fn new(tokens: Vec<Token>, debugger: OmniDebugger) -> Self {
        let cursor = TokenCursor::new(tokens, debugger.clone());
        Self {
            cursor,
            debugger,
        }
    }

    /// Inspects the current token at the cursor without advancing.
    #[inline]
    pub fn peek(&self) -> &Token {
        self.cursor.peek()
    }

    /// Inspects a future token at `offset` positions ahead of the cursor.
    #[inline]
    pub fn peek_at(&self, offset: usize) -> &Token {
        self.cursor.peek_at(offset)
    }

    /// Advances the cursor by one token and returns the consumed token.
    #[inline]
    pub fn advance(&mut self) -> Token {
        self.cursor.advance()
    }

    /// Checks if the current token matches the expected token type without consuming it.
    #[inline]
    pub fn check(&self, expected: &TokenType) -> bool {
        self.cursor.check(expected)
    }
```

```
    /// Checks if a lookahead token at `offset` matches the expected token type.
    #[inline]
    pub fn check_at(&self, offset: usize, expected: &TokenType) -> bool {
        self.cursor.check_at(offset, expected)
    }

    /// Conditionally consumes the current token if it matches the expected type.
    #[inline]
    pub fn match_token(&mut self, expected: &TokenType) -> bool {
        self.cursor.match_token(expected)
    }

    /// Returns `true` if the cursor has reached the end-of-file token.
    #[inline]
    pub fn is_at_end(&self) -> bool {
        self.cursor.is_at_end()
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Grammar Coordination: Program, Statements & Expressions )
// ============================================================================

impl Parser {
    /// Parses an entire source reel or tape into a unified [`ProgramNode`].
    ///
    /// Iterates through sequential top-level statements until [`TokenType::Eof`],
    /// synchronizes debugger state, and verifies that all opened boundaries closed cleanly.
    pub fn parse_program(&mut self) -> Result<ProgramNode, String> {
        let mut engine = GrammarEngine::new(&mut self.cursor);
        let res = engine.parse_program();
        self.debugger = self.cursor.debugger.clone();
        res
    }

    /// Parses a single statement or top-level structural declaration.
    pub fn parse_statement(&mut self) -> Result<ASTNode, String> {
        let mut engine = GrammarEngine::new(&mut self.cursor);
        let res = engine.parse_statement();
        self.debugger = self.cursor.debugger.clone();
        res
    }

    /// Entry point for expression hierarchy parsing.
    pub fn parse_expression(&mut self) -> Result<ASTNode, String> {
        let mut engine = GrammarEngine::new(&mut self.cursor);
        let res = engine.parse_expression();
        self.debugger = self.cursor.debugger.clone();
        res
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Parsing Invariant Verification, Boundary Integrity Proofs & Test Oracles )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;
    use crate::diagnostics::logger::DiagnosticLogger;

    #[test]
    fn test_parser_leading_equals_module_declaration() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 5);
        let src = "= [Module]: < Lexicon { Types } > & [ Noun { Character } ] { [ Version ]: { \"1.0.0\" } }";
        let mut lexer = crate::parser::lexer::Lexer::new(src, logger);
        let tokens = lexer.tokenize().expect("Lexing must succeed");
        let mut parser = Parser::new(tokens, debugger);
        let prog = parser.parse_program();
        assert!(prog.is_ok(), "Module declaration starting with '=' must parse cleanly: {:?}", prog.err());
    }

    #[test]
    fn test_parser_ranked_asymmetric_boundaries() {
```

```
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 5);
        let src = "{ Input ] --> [ Output }";
        let mut lexer = crate::parser::lexer::Lexer::new(src, logger);
        let tokens = lexer.tokenize().expect("Lexing must succeed");
        let mut parser = Parser::new(tokens, debugger);
        let prog = parser.parse_program();
        assert!(prog.is_ok(), "Asymmetric boundaries {{ ... ] and [ ... }} must parse cleanly: {:?}", prog.err());
    }

    #[test]
    fn test_parser_adversative_caveat_interlock() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 5);
        let src = "Action [ +1 ] ! [ Guard = +1 ]";
        let mut lexer = crate::parser::lexer::Lexer::new(src, logger);
        let tokens = lexer.tokenize().expect("Lexing must succeed");
        let mut parser = Parser::new(tokens, debugger);
        let prog = parser.parse_program();
        assert!(prog.is_ok(), "Adversative caveat ! must parse cleanly: {:?}", prog.err());
    }

    #[test]
    fn test_parser_character_omni_source_reel_full_parse() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 100);
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.omni");
        assert!(path.exists(), "character.omni must exist at resolved track path");
        let src = std::fs::read_to_string(&path).expect("character.omni must be readable");
        let mut lexer = crate::parser::lexer::Lexer::new(&src, logger);
        let tokens = lexer.tokenize().expect("character.omni lexing must succeed");
        let mut parser = Parser::new(tokens, debugger);
        let prog = parser.parse_program();
        assert!(prog.is_ok(), "character.omni must parse cleanly into AST: {:?}", prog.err());
        let program_node = prog.unwrap();
        assert!(!program_node.statements.is_empty(), "character.omni AST must contain parsed statements");
    }

    #[test]
    fn test_parser_svo_verbs_and_composed_locators() {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 10);
        let src = "[ ROLL 4 ] : [ Project ] : [ Character` } --> PROJECT [ Target ] ::@ 0x01";
        let mut lexer = crate::parser::lexer::Lexer::new(src, logger);
        let tokens = lexer.tokenize().expect("Lexing must succeed");
        let mut parser = Parser::new(tokens, debugger);
        let prog = parser.parse_program();
        assert!(prog.is_ok(), "SVO verb PROJECT and composed locator ::@ 0x01 must parse cleanly: {:?}", prog.err());
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/lexical/lexer.rs RS 678 LINES 2858 WORDS 32.0 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Lexer Identity, Ontological TokenType Re-exports & Morphemic Invariants )
// ============================================================================
//! # OmniCode Lexical Tokenizer (`lexer.rs`)
//!
//! Transforms raw source text reels into structured [`Token`] streams, enforcing
//! the physical and ontological boundary invariants of OmniCode.
//!
//! ## Scriptural Grounding:
//! - **Exodus 31:18**: *"And he gave unto Moses, when he had made an end of communing
//!   with him upon mount Sinai, two tables of testimony, tables of stone, written with
//!   the finger of God."*
//!   The Lexer transcribes divine mathematical axioms from the source reel into
//!   discrete, uncorrupted tokens of stone.
//! - **Proverbs 24:27**: *"Prepare thy work without, and make it fit for thyself in
//!   the field; and afterwards build thine house."*
//!   Morphemic tokens are meticulously prepared and validated in isolation before
//!   the syntactic house is erected by the grammar engine.
//! - **Proverbs 22:28**: *"Remove not the ancient landmark, which thy fathers have set."*
//!   Preserves the 6 ranked boundary landmarks and invariant operators across all reels.
//!
//! ## Architectural Foundations:
//! 1. **Modular Sub-Engine Structure**:
//!    - Token definitions and morphemes reside in [`crate::parser::tokens`].
//!    - Universal character streaming and coordinate tracking reside in [`crate::parser::char_stream`].
//!    - Language-specific compounding, state-primes, and boundary logic reside here.
//!
//! 2. **The 6 Ranked Morphological Boundary Delimiters**:
//!    - Rank 1 `< ... >`: Invariant Axiom (Absolute Immutable Law & Ontological Truth)
//!    - Rank 2 `[ ... ]`: Denotation (Concrete State Container & Evaluation Block)
//!    - Rank 3 `{ ... }`: Connotation (Context, Environment & Payload Attributes)
//!    - Rank 4 `[ ... }`: Inside-Out Emergence (Output Projection to External Universe)
//!    - Rank 5 `{ ... ]`: Outside-In Imposition (Input Capture from External Universe)
//!    - Rank 6 `( ... )`: Intent Aside (First-Class Cognitive Teleology & Rationale)
//!
//! 3. **First-Class Machine Intent**:
//!    Asides enclosed in `( ... )` are **never** discarded as trivia or comments.
//!    They are preserved in the token stream and lowered into the AST as persistent
//!    teleological metadata nodes, recording the *why* of every machine state.
//!
//! 4. **Causal State-Primes (`Character'`, ```Character` ```)**:
//!    Discrete clock cycles advance an entity from state $S$ to state $S'$. The state-prime
//!    operator is tokenized as part of the identifier, designating causality without collision.
//!
//! 5. **Dual-Substrate Arithmetic & Hardware Locators**:
//!    Supports balanced-ternary literals (`+1:PROCEED`, `-1:ROLLBACK`, `0:AWAIT`),
//!    hexadecimal hardware addresses (`0x01`, `0x000B8000`), hardware locators (`::@`),
//!    SVO natural arithmetic (`plus`, `minus`, `times`, `div`), and agentic pronouns.

pub use crate::parser::tokens::{SourceLocation, Span, Token, TokenType};
pub use crate::parser::char_stream::{CharCheckpoint, CharCursor};

use crate::core::boundaries::BoundaryType;
use crate::core::operators::{AffixPosition, Polarity};
use crate::core::ternary::QuadState;
use crate::core::linguistics::PronounKind;
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType};


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Lexer State Structure & Coordinator Initializer )
// ============================================================================

/// State container for source text reel tokenization.
pub struct Lexer<'a> {
    /// Universal character stream cursor.
    cursor: CharCursor<'a>,
    /// Depth counter for nested square brackets `[...]`.
    pub bracket_depth: usize,
    /// Diagnostic logger recording failpoint telemetry and lexical events.
    pub logger: DiagnosticLogger,
}
```

```rust

impl<'a> Lexer<'a> {
    /// Constructs a new [`Lexer`] over an OmniCode source reel string.
    ///
    /// # Arguments
    /// * `source` - Complete OmniCode UTF-8 source string.
    /// * `logger` - Diagnostic logger for telemetry and failpoint recording.
    pub fn new(source: &'a str, logger: DiagnosticLogger) -> Self {
        Self {
            cursor: CharCursor::new(source),
            bracket_depth: 0,
            logger,
        }
    }

    /// Read-only reference to the internal character cursor.
    #[inline]
    pub fn cursor(&self) -> &CharCursor<'a> {
        &self.cursor
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Tokenization Transformation Pipeline & Modular Scanning Engines )
// ============================================================================
impl<'a> Lexer<'a> {
    /// Tokenizes the entire source reel into a complete [`Token`] stream.
    ///
    /// Evaluates sequential lexical passes:
    /// 1. Skip whitespace and `#` comments.
    /// 2. First-class Intent Asides `( ... )`.
    /// 3. Multi-character compound operators and directional vectors.
    /// 4. Prefixed polarity identifiers (`+Word`, `-Word`).
    /// 5. Single-character morphological boundaries and operators.
    /// 6. Numeric literals (Hexadecimal and Decimal).
    /// 7. Identifiers, State-Primes (`Word'`, ```Word` ```), Postfix Polarities, Pronouns, and Keywords.
    pub fn tokenize(&mut self) -> Result<Vec<Token>, String> {
        let mut tokens = Vec::new();

        while !self.cursor.is_at_end() {
            self.cursor.skip_whitespace_and_comments('#');
            if self.cursor.is_at_end() {
                break;
            }

            let cur_line = self.cursor.line();
            let cur_col = self.cursor.col();

            // ---------------------------------------------------------------
            // 1. FIRST-CLASS INTENT ASIDES: ( ... )
            // ---------------------------------------------------------------
            if self.cursor.peek() == Some('(') {
                self.cursor.advance();
                tokens.push(self.read_aside(cur_line, cur_col)?);
                continue;
            }

            // ---------------------------------------------------------------
            // 2. MULTI-CHARACTER COMPOUND OPERATORS, VECTORS & LOCATORS
            // ---------------------------------------------------------------
            if let Some(tok) = self.match_compound_operators(cur_line, cur_col) {
                tokens.push(tok);
                continue;
            }

            // ---------------------------------------------------------------
            // 3. PREFIXED POLARITY IDENTIFIERS: +Word (Abstract), -Word (Concrete)
            // ---------------------------------------------------------------
            if (self.cursor.peek() == Some('+') || self.cursor.peek() == Some('-'))
                && self.cursor.peek_matches(1, |c| c.is_alphabetic() || c == '_')
            {
                tokens.push(self.read_prefixed_polarity(cur_line, cur_col));
                continue;
            }

            // ---------------------------------------------------------------
```

```
            // 4. BOUNDARY DELIMITERS, NUMBERS, STRINGS & IDENTIFIERS
            // ------------------------------------------------------------------
            let ch = self.cursor.peek().unwrap();
            match ch {
                // Morphological Boundaries: [ ] { } < >
                '[' | ']' | '{' | '}' | '<' | '>' => {
                    tokens.push(self.read_boundary_delimiter(ch, tokens.last(), cur_line, cur_col));
                }

                // Single-Character Operators & Connectives
                '=' | '!' | ':' | '|' | '?' | '@' | '%' | '&' | '+' | '-' | '*' => {
                    tokens.push(self.read_single_char_operator(ch, cur_line, cur_col));
                }

                // String & Character Literals ('...' or "...")
                '"' | '\'' => {
                    self.cursor.advance();
                    tokens.push(self.read_string_or_char(ch, cur_line, cur_col)?);
                }

                // Statement and Element Separators
                ';' | ',' => {
                    self.cursor.advance();
                }

                // Numeric Literals (Hexadecimal `0x...` or Decimal)
                _ if ch.is_ascii_digit() => {
                    tokens.push(self.read_number(ch, cur_line, cur_col)?);
                }

                // Identifiers, State-Primes, Pronouns, and Keywords
                _ if ch.is_alphabetic() || ch == '_' => {
                    tokens.push(self.read_identifier_or_keyword(cur_line, cur_col));
                }

                // Unrecognized Character Anomaly
                other => {
                    let msg = format!("Unexpected character '{other}' at L{cur_line}:C{cur_col}");
                    self.logger.failpoint(FailpointType::SyntaxAnomaly, "LEXER", &msg);
                    return Err(format!("FAILPOINT: {msg}"));
                }
            }
        }

        tokens.push(Token::eof(self.cursor.line(), self.cursor.col()));
        Ok(tokens)
    }

    /// Matches multi-character compound operators, directional vectors, casts, and locators
    /// using zero-allocation matching via [`CharCursor`].
    fn match_compound_operators(&mut self, cur_line: usize, cur_col: usize) -> Option<Token> {
        // Freeform Boundary Casts: [+], [-], <+>, <->, {+}, {-}
        if self.cursor.match_str("[+]") {
            return Some(Token::new(TokenType::BoundaryCast { boundary: BoundaryType::Denotation, polarity:
Polarity::Abstract }, "[+]", cur_line, cur_col));
        }
        if self.cursor.match_str("[-]") {
            return Some(Token::new(TokenType::BoundaryCast { boundary: BoundaryType::Denotation, polarity:
Polarity::Concrete }, "[-]", cur_line, cur_col));
        }
        if self.cursor.match_str("<+>") {
            return Some(Token::new(TokenType::BoundaryCast { boundary: BoundaryType::Invariant, polarity:
Polarity::Abstract }, "<+>", cur_line, cur_col));
        }
        if self.cursor.match_str("<->") {
            return Some(Token::new(TokenType::BoundaryCast { boundary: BoundaryType::Invariant, polarity:
Polarity::Concrete }, "<->", cur_line, cur_col));
        }
        if self.cursor.match_str("{+}") {
            return Some(Token::new(TokenType::BoundaryCast { boundary: BoundaryType::Connotation, polarity:
Polarity::Abstract }, "{+}", cur_line, cur_col));
        }
        if self.cursor.match_str("{-}") {
            return Some(Token::new(TokenType::BoundaryCast { boundary: BoundaryType::Connotation, polarity:
Polarity::Concrete }, "{-}", cur_line, cur_col));
        }

        // 3-Character Directional Vectors, Equivalence & Hardware Locator
```

```
        if self.cursor.match_str("-->") {
            return Some(Token::new(TokenType::ActUpon, "-->", cur_line, cur_col));
        }
        if self.cursor.match_str("<--") {
            return Some(Token::new(TokenType::ActAgain, "<--", cur_line, cur_col));
        }
        if self.cursor.match_str("--^") || self.cursor.match_str("^--") {
            return Some(Token::new(TokenType::BuildsTo, "^--", cur_line, cur_col));
        }
        if self.cursor.match_str("--v") || self.cursor.match_str("v--") {
            return Some(Token::new(TokenType::BreaksDownFrom, "v--", cur_line, cur_col));
        }
        if self.cursor.match_str("<=>") {
            return Some(Token::new(TokenType::InvariantEquivalence, "<=>", cur_line, cur_col));
        }
        if self.cursor.match_str("::@") {
            return Some(Token::new(TokenType::HardwareLocator, "::@", cur_line, cur_col));
        }

        // 2-Character Directional Vectors, Relations & Connectives
        if self.cursor.match_str("->") {
            return Some(Token::new(TokenType::ActUpon, "->", cur_line, cur_col));
        }
        if self.cursor.match_str("<-") {
            return Some(Token::new(TokenType::ActAgain, "<-", cur_line, cur_col));
        }
        if self.cursor.match_str("-^") {
            return Some(Token::new(TokenType::BuildsTo, "-^", cur_line, cur_col));
        }
        if self.cursor.match_str("-v") {
            return Some(Token::new(TokenType::BreaksDownFrom, "-v", cur_line, cur_col));
        }
        if self.cursor.match_str("::") {
            return Some(Token::new(TokenType::And, "::", cur_line, cur_col));
        }
        if self.cursor.match_str("!=") {
            return Some(Token::new(TokenType::IsNot, "!=", cur_line, cur_col));
        }
        if self.cursor.match_str("=!") {
            return Some(Token::new(TokenType::ToBeNot, "=!", cur_line, cur_col));
        }
        if self.cursor.match_str("~=") {
            return Some(Token::new(TokenType::ToBeLike, "~=", cur_line, cur_col));
        }
        if self.cursor.match_str("=~") {
            return Some(Token::new(TokenType::ToBeLike, "=~", cur_line, cur_col));
        }
        if self.cursor.match_str("~!") {
            return Some(Token::new(TokenType::But, "~!", cur_line, cur_col));
        }
        if self.cursor.match_str("<=") {
            return Some(Token::new(TokenType::LessEqual, "<=", cur_line, cur_col));
        }
        if self.cursor.match_str(">=") {
            return Some(Token::new(TokenType::GreaterEqual, ">=", cur_line, cur_col));
        }
        if self.cursor.match_str("--") {
            return Some(Token::new(TokenType::Not, "--", cur_line, cur_col));
        }

        // Diagonal Scale Modulations & 2D Quadrants
        if self.cursor.match_str("/-^") {
            return Some(Token::new(TokenType::Major, "/-^", cur_line, cur_col));
        }
        if self.cursor.match_str("\\-v") {
            return Some(Token::new(TokenType::Minor, "\\-v", cur_line, cur_col));
        }
        if self.cursor.match_str("\\-^") {
            return Some(Token::new(TokenType::Great, "\\-^", cur_line, cur_col));
        }
        if self.cursor.match_str("/-v") {
            return Some(Token::new(TokenType::Small, "/-v", cur_line, cur_col));
        }
        if self.cursor.match_str("/^") {
            return Some(Token::new(TokenType::UpperRight, "/^", cur_line, cur_col));
        }
        if self.cursor.match_str("\\v") {
            return Some(Token::new(TokenType::LowerRight, "\\v", cur_line, cur_col));
```

```
        }
        if self.cursor.match_str("^\\") {
            return Some(Token::new(TokenType::UpperLeft, "^\\", cur_line, cur_col));
        }
        if self.cursor.match_str("v/") {
            return Some(Token::new(TokenType::LowerLeft, "v/", cur_line, cur_col));
        }
        if self.cursor.match_str("..") {
            return Some(Token::new(TokenType::DotDot, "..", cur_line, cur_col));
        }

        // Signed Balanced-Ternary Literals: +1, -1
        if self.cursor.match_str("+1") {
            return Some(Token::new(TokenType::Ternary(QuadState::Proceed), "+1", cur_line, cur_col));
        }
        if self.cursor.match_str("-1") {
            return Some(Token::new(TokenType::Ternary(QuadState::Rollback), "-1", cur_line, cur_col));
        }

        None
    }

    /// Scans a first-class teleological Intent Aside enclosed in `( ... )`.
    fn read_aside(&mut self, start_line: usize, start_col: usize) -> Result<Token, String> {
        let mut depth = 1;
        let mut content = String::new();

        while let Some(ch) = self.cursor.advance() {
            match ch {
                '(' => {
                    depth += 1;
                    content.push(ch);
                }
                ')' => {
                    depth -= 1;
                    if depth == 0 {
                        let text = content.trim().to_string();
                        self.logger.log("ASIDE", format!("Captured Aside: ({text})"));
                        return Ok(Token::new(TokenType::Aside(text), format!("({content})"), start_line, start_col));
                    }
                    content.push(ch);
                }
                _ => content.push(ch),
            }
        }

        let msg = format!("Unclosed Aside '(' opened at L{start_line}:C{start_col} reaches EOF");
        self.logger.failpoint(FailpointType::BoundaryLeak, "LEXER", &msg);
        Err(format!("FAILPOINT [UnclosedAside]: {msg}"))
    }

    /// Scans a quoted string literal (`"..."`) or atomic character literal (`'...'`).
    fn read_string_or_char(&mut self, quote: char, start_line: usize, start_col: usize) -> Result<Token, String> {
        let mut s = String::new();
        while let Some(ch) = self.cursor.advance() {
            if ch == quote {
                // If single quote with single character, recognize as atomic Character glyph
                if quote == '\'' && s.chars().count() == 1 {
                    let c = s.chars().next().unwrap();
                    return Ok(Token::new(TokenType::CharLit(c), format!("'{s}'"), start_line, start_col));
                }
                return Ok(Token::new(TokenType::StringLit(s.clone()), format!("\"{s}\""), start_line, start_col));
            } else if ch == '\\' {
                if let Some(esc) = self.cursor.advance() {
                    match esc {
                        'n' => s.push('\n'),
                        'r' => s.push('\r'),
                        't' => s.push('\t'),
                        '\\' => s.push('\\'),
                        '\'' => s.push('\''),
                        '"' => s.push('"'),
                        '0' => s.push('\0'),
                        other => s.push(other),
                    }
                }
            } else {
                s.push(ch);
            }
```

```
        }
        let msg = format!("Unclosed literal opened at L{start_line}:C{start_col} reaches EOF");
        self.logger.failpoint(FailpointType::SyntaxAnomaly, "LEXER", &msg);
        Err(format!("FAILPOINT: {msg}"))
    }

    /// Scans a prefixed polarity identifier (`+Word` or `-Word`).
    fn read_prefixed_polarity(&mut self, cur_line: usize, cur_col: usize) -> Token {
        let sign = self.cursor.advance().unwrap();
        let polarity = if sign == '+' { Polarity::Abstract } else { Polarity::Concrete };
        let ident = self.cursor.consume_while(|c| c.is_alphanumeric() || c == '_' || c == '.' || c == '?');
        let lexeme = format!("{}{}", sign, ident);
        Token::new(
            TokenType::AffixedIdent {
                name: ident,
                polarity,
                position: AffixPosition::Prefix,
            },
            lexeme,
            cur_line,
            cur_col,
        )
    }

    /// Scans a single-character morphological boundary delimiter.
    fn read_boundary_delimiter(&mut self, ch: char, last_token: Option<&Token>, cur_line: usize, cur_col: usize) -> Token
{
        self.cursor.advance();
        match ch {
            '[' => {
                self.bracket_depth += 1;
                Token::new(TokenType::LBracket, "[", cur_line, cur_col)
            }
            ']' => {
                self.bracket_depth = self.bracket_depth.saturating_sub(1);
                Token::new(TokenType::RBracket, "]", cur_line, cur_col)
            }
            '{' => Token::new(TokenType::LBrace, "{", cur_line, cur_col),
            '}' => Token::new(TokenType::RBrace, "}", cur_line, cur_col),
            '<' => {
                // Canon 3 Disambiguation: Casted context comparison [ { A } < { B } ]
                if self.bracket_depth > 0
                    && last_token.map_or(false, |t| t.token_type == TokenType::RBrace)
                    && self.cursor.peek_next_non_whitespace(0) == Some('{')
                {
                    Token::new(TokenType::LessThan, "<", cur_line, cur_col)
                } else {
                    Token::new(TokenType::LAngle, "<", cur_line, cur_col)
                }
            }
            '>' => {
                // Canon 3 Disambiguation: Casted context comparison [ { A } > { B } ]
                if self.bracket_depth > 0
                    && last_token.map_or(false, |t| t.token_type == TokenType::RBrace)
                    && self.cursor.peek_next_non_whitespace(0) == Some('{')
                {
                    Token::new(TokenType::GreaterThan, ">", cur_line, cur_col)
                } else {
                    Token::new(TokenType::RAngle, ">", cur_line, cur_col)
                }
            }
            _ => unreachable!(),
        }
    }

    /// Scans a single-character operator or connective.
    fn read_single_char_operator(&mut self, ch: char, cur_line: usize, cur_col: usize) -> Token {
        self.cursor.advance();
        match ch {
            '=' => Token::new(TokenType::Is, "=", cur_line, cur_col),
            '!' => Token::new(TokenType::But, "!", cur_line, cur_col),
            ':' => Token::new(TokenType::InRelationTo, ":", cur_line, cur_col),
            '|' => Token::new(TokenType::Or, "|", cur_line, cur_col),
            '?' => Token::new(TokenType::Infer, "?", cur_line, cur_col),
            '@' => Token::new(TokenType::At, "@", cur_line, cur_col),
            '%' => Token::new(TokenType::Percent, "%", cur_line, cur_col),
            '&' => Token::new(TokenType::BoundPair, "&", cur_line, cur_col),
            '+' => Token::new(TokenType::Plus, "+", cur_line, cur_col),
```

```
            '-' => Token::new(TokenType::Minus, "-", cur_line, cur_col),
            '*' => Token::new(TokenType::Times, "*", cur_line, cur_col),
            _ => unreachable!(),
        }
    }

    /// Scans numeric literals, including hexadecimal (`0x...`), standalone ternary zero `0`,
    /// and decimal integers/floats.
    fn read_number(&mut self, ch: char, cur_line: usize, cur_col: usize) -> Result<Token, String> {
        // Hexadecimal hardware address locator: 0x01, 0x000B8000
        if ch == '0' && (self.cursor.peek_at(1) == Some('x') || self.cursor.peek_at(1) == Some('X')) {
            self.cursor.advance(); // consume '0'
            self.cursor.advance(); // consume 'x' or 'X'
            let hex_str = self.cursor.consume_while(|c| c.is_ascii_hexdigit());
            let val = u64::from_str_radix(&hex_str, 16)
                .map_err(|e| format!("FAILPOINT: Invalid hex number '0x{hex_str}': {e}"))?;
            return Ok(Token::new(TokenType::Number(val as f64), format!("0x{hex_str}"), cur_line, cur_col));
        }

        // Binary bit-plane pattern: 0b00, 0b01, 0b10, 0b11, 0b0101_1010
        if ch == '0' && (self.cursor.peek_at(1) == Some('b') || self.cursor.peek_at(1) == Some('B')) {
            self.cursor.advance(); // consume '0'
            self.cursor.advance(); // consume 'b' or 'B'
            let bin_str = self.cursor.consume_while(|c| c == '0' || c == '1' || c == '_');
            let clean_bin: String = bin_str.chars().filter(|&c| c != '_').collect();
            let val = u64::from_str_radix(&clean_bin, 2)
                .map_err(|e| format!("FAILPOINT: Invalid binary number '0b{bin_str}': {e}"))?;
            return Ok(Token::new(TokenType::Number(val as f64), format!("0b{bin_str}"), cur_line, cur_col));
        }

        // Standalone ternary zero `0` representing Await / Equilibrium
        if ch == '0' && !self.cursor.peek_matches(1, |c| c.is_alphanumeric() || c == '.') {
            self.cursor.advance();
            return Ok(Token::new(TokenType::Ternary(QuadState::Await), "0", cur_line, cur_col));
        }

        // Decimal integer or floating point number
        let num_str = self.cursor.consume_while(|c| c.is_ascii_digit() || c == '.');
        let val: f64 = num_str.parse().map_err(|e| format!("FAILPOINT: Invalid number '{num_str}': {e}"))?;
        Ok(Token::new(TokenType::Number(val), num_str, cur_line, cur_col))
    }

    /// Scans identifiers, SVO verbs, state-primes (`Name'`, ```Name` ```), postfix polarities,
    /// pronouns, and grammatical keywords.
    fn read_identifier_or_keyword(&mut self, cur_line: usize, cur_col: usize) -> Token {
        // Accumulate identifier characters, including state-prime (`) and ticks (')
        let ident = self.cursor.consume_while(|c| {
            c.is_alphanumeric() || c == '_' || c == '.' || c == '?' || c == '\'' || c == '`'
        });

        // Check for attached Postfix Polarity: Word+ (Abstract), Word- (Concrete)
        let mut postfix_polarity = None;
        if self.cursor.peek() == Some('+') {
            let next_after = self.cursor.peek_at(1);
            if next_after.map_or(true, |c| {
                c.is_whitespace() || c == ']' || c == '}' || c == '>' || c == ')' || c == ',' || c == ';' || c == '!' ||
c == '@' || c == ':'
            }) {
                self.cursor.advance();
                postfix_polarity = Some(Polarity::Abstract);
            }
        } else if self.cursor.peek() == Some('-') {
            let next_after = self.cursor.peek_at(1);
            if next_after.map_or(true, |c| {
                c != '>' && c != '-' && c != '^' && c != 'v' && (
                    c.is_whitespace() || c == ']' || c == '}' || c == '>' || c == ')' || c == ',' || c == ';' || c == '!'
|| c == '@' || c == ':'
                )
            }) {
                self.cursor.advance();
                postfix_polarity = Some(Polarity::Concrete);
            }
        }

        if let Some(polarity) = postfix_polarity {
            let lexeme = format!("{}{}", ident, polarity.symbol());
            return Token::new(
                TokenType::AffixedIdent {
```

```
                    name: ident,
                    polarity,
                    position: AffixPosition::Postfix,
                },
                lexeme,
                cur_line,
                cur_col,
            );
        }

        // Disambiguate keywords, pronouns, ternary states, and general identifiers
        match ident.to_lowercase().as_str() {
            "null" | "void" | "halt" | "stop" => Token::new(TokenType::Ternary(QuadState::Halt), ident, cur_line,
cur_col),
            "yes" | "proceed" | "play" => Token::new(TokenType::Ternary(QuadState::Proceed), ident, cur_line, cur_col),
            "ok" | "await" | "pause" => Token::new(TokenType::Ternary(QuadState::Await), ident, cur_line, cur_col),
            "no" | "rollback" | "rewind" => Token::new(TokenType::Ternary(QuadState::Rollback), ident, cur_line,
cur_col),
            "plus" => Token::new(TokenType::Plus, ident, cur_line, cur_col),
            "minus" => Token::new(TokenType::Minus, ident, cur_line, cur_col),
            "times" => Token::new(TokenType::Times, ident, cur_line, cur_col),
            "div" => Token::new(TokenType::Div, ident, cur_line, cur_col),
            "who" => Token::new(TokenType::Who, ident, cur_line, cur_col),
            "what" => Token::new(TokenType::What, ident, cur_line, cur_col),
            "when" => Token::new(TokenType::When, ident, cur_line, cur_col),
            "where" => Token::new(TokenType::Where, ident, cur_line, cur_col),
            "why" => Token::new(TokenType::Why, ident, cur_line, cur_col),
            "how" => Token::new(TokenType::How, ident, cur_line, cur_col),
            _ => {
                if let Some(p) = PronounKind::from_keyword(&ident) {
                    Token::new(TokenType::Pronoun(p), ident, cur_line, cur_col)
                } else {
                    Token::new(TokenType::Identifier(ident.clone()), ident, cur_line, cur_col)
                }
            }
        }
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Lexical Verification Proofs, Boundary Integrity & Test Oracles )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_lexing_pronouns_and_reflexives() {
        let logger = DiagnosticLogger::new(false);
        let mut lexer = Lexer::new("I you it they myself itself my your _", logger);
        let tokens = lexer.tokenize().expect("Lexing pronouns");

        assert_eq!(tokens[0].token_type, TokenType::Pronoun(PronounKind::I));
        assert_eq!(tokens[1].token_type, TokenType::Pronoun(PronounKind::You));
        assert_eq!(tokens[2].token_type, TokenType::Pronoun(PronounKind::It));
        assert_eq!(tokens[3].token_type, TokenType::Pronoun(PronounKind::They));
        assert_eq!(tokens[4].token_type, TokenType::Pronoun(PronounKind::Myself));
        assert_eq!(tokens[5].token_type, TokenType::Pronoun(PronounKind::Itself));
        assert_eq!(tokens[6].token_type, TokenType::Pronoun(PronounKind::My));
        assert_eq!(tokens[7].token_type, TokenType::Pronoun(PronounKind::Your));
        assert_eq!(tokens[8].token_type, TokenType::Pronoun(PronounKind::Wildcard));
    }

    #[test]
    fn test_lexing_hexadecimal_numbers_and_locators() {
        let logger = DiagnosticLogger::new(false);
        let mut lexer = Lexer::new("0x01 ::@ 0x000B8000", logger);
        let tokens = lexer.tokenize().expect("Lexing hex numbers");

        assert_eq!(tokens[0].token_type, TokenType::Number(1.0));
        assert!(tokens[1].is_hardware_locator());
        assert_eq!(tokens[1].token_type, TokenType::HardwareLocator);
        assert_eq!(tokens[1].lexeme, "::@");
        assert_eq!(tokens[2].token_type, TokenType::Number(0xB8000 as f64));
    }

```

```
    #[test]
    fn test_lexing_state_primes_and_svo_verbs() {
        let logger = DiagnosticLogger::new(false);
        let mut lexer = Lexer::new("Character` --> INSCRIBE [ Cell ] = [ Character` ]", logger);
        let tokens = lexer.tokenize().expect("Lexing state primes and verbs");

        assert_eq!(tokens[0].lexeme, "Character`");
        assert!(tokens[0].is_state_prime());
        assert_eq!(tokens[1].token_type, TokenType::ActUpon);
        assert_eq!(tokens[2].lexeme, "INSCRIBE");
        assert_eq!(tokens[3].token_type, TokenType::LBracket);
        assert_eq!(tokens[4].lexeme, "Cell");
        assert_eq!(tokens[5].token_type, TokenType::RBracket);
        assert_eq!(tokens[6].token_type, TokenType::Is);
        assert_eq!(tokens[7].token_type, TokenType::LBracket);
        assert_eq!(tokens[8].lexeme, "Character`");
        assert_eq!(tokens[9].token_type, TokenType::RBracket);
    }

    #[test]
    fn test_lexing_relational_bounds_and_transduction() {
        let logger = DiagnosticLogger::new(false);
        let mut lexer = Lexer::new("65 <=> [ -1, +1, +1, -1, +1 ]", logger);
        let tokens = lexer.tokenize().expect("Lexing transduction and trits");

        assert_eq!(tokens[0].token_type, TokenType::Number(65.0));
        assert_eq!(tokens[1].token_type, TokenType::InvariantEquivalence);
        assert_eq!(tokens[2].token_type, TokenType::LBracket);
        assert_eq!(tokens[3].token_type, TokenType::Ternary(QuadState::Rollback));
        assert_eq!(tokens[4].token_type, TokenType::Ternary(QuadState::Proceed));
    }

    #[test]
    fn test_lexing_single_char_glyph_and_string() {
        let logger = DiagnosticLogger::new(false);
        let mut lexer = Lexer::new("'A' \"Hello\"", logger);
        let tokens = lexer.tokenize().expect("Lexing glyph and string");

        assert_eq!(tokens[0].token_type, TokenType::CharLit('A'));
        assert_eq!(tokens[1].token_type, TokenType::StringLit("Hello".into()));
    }

    #[test]
    fn test_lexing_character_omni_source_reel_token_stream() {
        let logger = DiagnosticLogger::new(false);
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.omni");
        assert!(path.exists(), "character.omni must exist");
        let src = std::fs::read_to_string(&path).expect("Read character.omni");
        let mut lexer = Lexer::new(&src, logger);
        let tokens = lexer.tokenize().expect("Tokenize character.omni");

        assert!(!tokens.is_empty(), "Token stream must not be empty");
        assert_eq!(tokens.last().unwrap().token_type, TokenType::Eof);
    }

    #[test]
    fn test_lexing_binary_numbers() {
        let logger = DiagnosticLogger::new(false);
        let mut lexer = Lexer::new("0b01 0b10 0b00 0b11 0b1010_0101", logger);
        let tokens = lexer.tokenize().expect("Lexing binary numbers");

        assert_eq!(tokens[0].token_type, TokenType::Number(1.0));
        assert_eq!(tokens[1].token_type, TokenType::Number(2.0));
        assert_eq!(tokens[2].token_type, TokenType::Number(0.0));
        assert_eq!(tokens[3].token_type, TokenType::Number(3.0));
        assert_eq!(tokens[4].token_type, TokenType::Number(165.0));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/lexical/mod.rs RS 20 LINES 95 WORDS 0.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Lexical Inscription & Token Vocabulary Subsystem )
// ============================================================================
//! # Lexical Inscription Subsystem
//!
//! Transforms raw source reels into structured [`Token`] streams, enforcing
//! the physical and ontological boundary invariants of OmniCode.
//!
//! ## Scriptural Grounding:
//! - **Exodus 31:18**: Divine inscription of characters on stone.
//! - **Proverbs 24:27**: Preparing tokens in the field before syntactic assembly.
//! - **Proverbs 22:28**: Preserving ancient morphemic boundaries and operators.

pub mod tokens;
pub mod lexer;

pub use tokens::{SourceLocation, Span, Token, TokenType};
pub use lexer::Lexer;
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/parser/lexical/tokens.rs RS 443 LINES 1748 WORDS 16.4 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Invariant Token Taxonomy, Morphemic Classifications & Source Spans )
// ============================================================================
//! # OmniCode Token Taxonomy (`tokens.rs`)
//!
//! Defines the ontological token vocabulary, morphemic classifications, and coordinate
//! spans emitted by the OmniCode lexical analyzer.
//!
//! ## Scriptural Grounding:
//! - **Exodus 31:18**: *"And he gave unto Moses, when he had made an end of communing
//!   with him upon mount Sinai, two tables of testimony, tables of stone, written with
//!   the finger of God."*
//!   Tokens are the immutable inscribed glyphs upon the silicon reel.
//! - **Proverbs 24:27**: *"Prepare thy work without, and make it fit for thyself in
//!   the field; and afterwards build thine house."*
//!   Lexical classification prepares every token in the field before the grammar engine
//!   builds the architectural syntactic house.
//! - **Proverbs 22:28**: *"Remove not the ancient landmark, which thy fathers have set."*
//!   Preserves ancient morphemic boundaries and structural rankings without corruption.
//!
//! ## Ontological Taxonomy:
//! 1. **Morphological Boundaries**: 6 ranked delimiters (`<...>`, `[...]`, `{...}`, `[...}`, `{...]`, `(...)`).
//! 2. **Agentic Pronouns**: Identity and capability pronouns (`I`, `you`, `it`, `they`, `myself`, etc.).
//! 3. **Polarities & Casts**: Concrete (`-`) and Abstract (`+`) affixes and boundary casts.
//! 4. **Directional Vectors**: 1D axial, 2D quadrant, and scale modulations (`-->`, `<--`, `^--`, `v--`, `/^`, `\v`).
//! 5. **Relations of Being**: Root assignment (`=`), similarity (`~=`), bounds (`<=`, `>=`), and bijective transduction (`<=>`).
//! 6. **Connectives & Locators**: Inseparable conjunctions (`::`), hardware locators (`::@`), adversarial caveats (`!`).
//! 7. **SVO Arithmetic**: Natural operation keywords (`plus`, `minus`, `times`, `div`).
//! 8. **Literals**: Balanced ternary (`+1`, `-1`, `0`, `NULL`), hex addresses (`0x...`), glyphs (`'A'`), strings (`"..."`).

use crate::core::boundaries::BoundaryType;
use crate::core::operators::{AffixPosition, Polarity};
use crate::core::ternary::QuadState;
use crate::core::linguistics::PronounKind;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Coordinates, Source Spans & Inscribed Token Unit )
// ============================================================================

/// Precise 1-indexed source reel location with byte offset tracking.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Default)]
pub struct SourceLocation {
    /// 1-indexed line number in source reel.
    pub line: usize,
    /// 1-indexed column number in source reel.
    pub col: usize,
    /// 0-indexed byte offset from start of source text.
    pub offset: usize,
}

impl SourceLocation {
    /// Creates a new source location coordinate.
    pub const fn new(line: usize, col: usize, offset: usize) -> Self {
        Self { line, col, offset }
    }
}

/// A contiguous source span from `start` coordinate to `end` coordinate.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Default)]
pub struct Span {
    /// Inclusive starting coordinate.
    pub start: SourceLocation,
    /// Inclusive ending coordinate.
    pub end: SourceLocation,
}

impl Span {
    /// Creates a new span spanning from `start` to `end`.
    pub const fn new(start: SourceLocation, end: SourceLocation) -> Self {
        Self { start, end }
```

```rust
    }

    /// Creates an atomic single-point span.
    pub const fn point(loc: SourceLocation) -> Self {
        Self { start: loc, end: loc }
    }
}

/// Inscribed token unit with source reel coordinate tracking.
#[derive(Debug, Clone, PartialEq)]
pub struct Token {
    /// Grammatical token classification.
    pub token_type: TokenType,
    /// Exact character sequence matched in the source text.
    pub lexeme: String,
    /// 1-indexed line coordinate in source reel.
    pub line: usize,
    /// 1-indexed column coordinate in source reel.
    pub col: usize,
}

impl Token {
    /// Constructs a new inscribed [`Token`].
    #[inline]
    pub fn new(token_type: TokenType, lexeme: impl Into<String>, line: usize, col: usize) -> Self {
        Self {
            token_type,
            lexeme: lexeme.into(),
            line,
            col,
        }
    }

    /// Constructs an End-Of-File sentinel token.
    #[inline]
    pub fn eof(line: usize, col: usize) -> Self {
        Self {
            token_type: TokenType::Eof,
            lexeme: String::new(),
            line,
            col,
        }
    }

    /// Returns `true` if this token is an EOF sentinel.
    #[inline]
    pub fn is_eof(&self) -> bool {
        self.token_type == TokenType::Eof
    }

    /// Checks if this token is a hardware locator (`::@`), supporting both first-class
    /// [`TokenType::HardwareLocator`] and legacy compounded [`TokenType::And`] with `"::@"`.
    #[inline]
    pub fn is_hardware_locator(&self) -> bool {
        self.token_type.is_hardware_locator(&self.lexeme)
    }

    /// Returns `true` if this token is an identifier designated as a state-prime (`Name'` or ```Name` ```).
    #[inline]
    pub fn is_state_prime(&self) -> bool {
        match &self.token_type {
            TokenType::Identifier(id) => TokenType::is_state_prime_name(id),
            _ => false,
        }
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Complete Ontological TokenType Hierarchy & Categorical Methods )
// ============================================================================

/// Complete taxonomy of grammatical tokens emitted by the Lexer.
#[derive(Debug, Clone, PartialEq)]
pub enum TokenType {
    // ------------------------------------------------------------------------
    // 1. Boundary Delimiters & First-Class Asides
```

```
    // --------------------------------------------------------------------------
    /// `[` Opening Denotation / Bottom-Up Emergence
    LBracket,
    /// `]` Closing Denotation / Top-Down Imposition
    RBracket,
    /// `{` Opening Connotation / Top-Down Imposition
    LBrace,
    /// `}` Closing Connotation / Bottom-Up Emergence
    RBrace,
    /// `<` Opening Invariant Axiom
    LAngle,
    /// `>` Closing Invariant Axiom
    RAngle,
    /// `( ... )` First-class Intent Aside capturing architectural rationale
    Aside(String),

    // --------------------------------------------------------------------------
    // 2. Grammatical Pronouns & Reflexives
    // --------------------------------------------------------------------------
    /// Agentic pronouns (`I`, `you`, `it`, `they`, `myself`, `itself`, `_`, etc.)
    Pronoun(PronounKind),

    // --------------------------------------------------------------------------
    // 3. Polarity & Boundary Cast Operators
    // --------------------------------------------------------------------------
    /// Affixed Identifiers (`+Word` Abstract, `-Word` Concrete, `Word+`, `Word-`)
    AffixedIdent {
        name: String,
        polarity: Polarity,
        position: AffixPosition,
    },
    /// Boundary Casts (`[+]`, `[-]`, `<+>`, `<->`, `{+}`, `{-}`)
    BoundaryCast {
        boundary: BoundaryType,
        polarity: Polarity,
    },

    // --------------------------------------------------------------------------
    // 4. Directional Vectors (Axial, Diagonal Scale Modulations & 2D Quadrants)
    // --------------------------------------------------------------------------
    /// `-->` / `->` Act Upon (Forward Vector / Transduction)
    ActUpon,
    /// `<--` / `<-` Act Again (Feedback / Rollback / Reverse Vector)
    ActAgain,
    /// `^--` / `--^` / `-^` Builds To (Ascension / Synthesis Vector)
    BuildsTo,
    /// `v--` / `--v` / `-v` Breaks Down From (Descension / Decomposition Vector)
    BreaksDownFrom,
    /// `/-^` Major scale up (Ascend forward)
    Major,
    /// `\-v` Minor scale down (Descend forward)
    Minor,
    /// `\-^` Great scale up (Ascend backward)
    Great,
    /// `/-v` Small scale down (Descend backward)
    Small,
    /// `/^` Quadrant I / Northeast
    UpperRight,
    /// `\v` Quadrant IV / Southeast
    LowerRight,
    /// `^\` Quadrant II / Northwest
    UpperLeft,
    /// `v/` Quadrant III / Southwest
    LowerLeft,

    // --------------------------------------------------------------------------
    // 5. Relations of Being & Invariant Transduction
    // --------------------------------------------------------------------------
    /// `=` Is (Relational Identity / Root Assignment)
    Is,
    /// `!=` Is Not (Relational Disparity / Not to be)
    IsNot,
    /// `=!` To Be Not (State Negation Predicate / Is not)
    ToBeNot,
    /// `~=` To Be Like (Fuzzy Affinity / Morphological Similarity)
    ToBeLike,
    /// `<=>` Invariant Equivalence (Bijective Dimensional Transduction: Bits <=> Trits)
    InvariantEquivalence,
```

```
    /// `<` Less Than (Casted context comparison)
    LessThan,
    /// `>` Greater Than (Casted context comparison)
    GreaterThan,
    /// `<=` Less Than or Equal To (Bound comparison: `{ Character` <= [ 1 Cell ] }`)
    LessEqual,
    /// `>=` Greater Than or Equal To (Bound comparison)
    GreaterEqual,

    // ---------------------------------------------------------------------
    // 6. Connectives, Conjunctions & Bindings
    // ---------------------------------------------------------------------
    /// `!` Adversative Caveat (A BUT B / Guard Interlock)
    But,
    /// `::` Inseparable Conjunction
    And,
    /// `::@` Dedicated Hardware Locator / MMIO Port Binding
    HardwareLocator,
    /// `:` In Relation To / Type Classification
    InRelationTo,
    /// `|` Disjunctive Alternative / Branching Choice
    Or,
    /// `--` Unary Negation (Not)
    Not,
    /// `&` Inseparable Twin Binding (A&B Hand-in-Hand)
    BoundPair,

    // ---------------------------------------------------------------------
    // 7. Prepositions, Locators, Ranges & Measures
    // ---------------------------------------------------------------------
    /// `@` Located At (MMIO Address / Stream Port)
    At,
    /// `..` Coordinate Range / Interval Span
    DotDot,
    /// `%` Proportion / Fullness / Fraction
    Percent,

    // ---------------------------------------------------------------------
    // 8. Cardinal Classification Clauses (The 5 W's and 1 H)
    // ---------------------------------------------------------------------
    /// `WHO` Agent / Actor / Identity
    Who,
    /// `WHAT` Substance / Entity / Vessel
    What,
    /// `WHEN` Cadence / Lifecycle / Trigger
    When,
    /// `WHERE` Spatial Site / Memory Partition / Scope
    Where,
    /// `WHY` Teleological Intent / Rationale / Axiom
    Why,
    /// `HOW` Causal Mechanism / Status / Roll Vector
    How,

    // ---------------------------------------------------------------------
    // 9. SVO Arithmetic Keywords
    // ---------------------------------------------------------------------
    /// `plus`, `add`
    Plus,
    /// `minus`, `sub`
    Minus,
    /// `times`, `mult`
    Times,
    /// `div`, `over`, `divided_by`
    Div,

    // ---------------------------------------------------------------------
    // 10. Inferential Operators & Terminals
    // ---------------------------------------------------------------------
    /// `?` Infer / Epistemic Query
    Infer,
    /// Balanced-Ternary Literals (`+1`, `-1`, `0`, `NULL`, `PROCEED`, `ROLLBACK`, `AWAIT`, `HALT`)
    Ternary(QuadState),
    /// Numeric Literals (Hexadecimal `0x...`, Decimal Integers, Floats)
    Number(f64),
    /// Atomic Character Glyph Literal (`'A'`, `'\n'`)
    CharLit(char),
    /// String Literals (`"..."`)
    StringLit(String),
```

```
    /// Generic Identifiers & SVO Verbs (supporting state-prime ``` ` ``` and `'`)
    Identifier(String),
    /// End-of-file sentinel
    Eof,
}

impl TokenType {
    /// Returns `true` if this token marks an opening morphological boundary (`[`, `{`, `<`).
    pub fn is_open_boundary(&self) -> bool {
        matches!(self, TokenType::LBracket | TokenType::LBrace | TokenType::LAngle)
    }

    /// Returns `true` if this token marks a closing morphological boundary (`]`, `}`, `>`).
    pub fn is_close_boundary(&self) -> bool {
        matches!(self, TokenType::RBracket | TokenType::RBrace | TokenType::RAngle)
    }

    /// Returns `true` if this token is any morphological boundary delimiter.
    pub fn is_boundary(&self) -> bool {
        self.is_open_boundary() || self.is_close_boundary()
    }

    /// Returns `true` if this token represents a directional vector.
    pub fn is_directional_vector(&self) -> bool {
        matches!(
            self,
            TokenType::ActUpon
                | TokenType::ActAgain
                | TokenType::BuildsTo
                | TokenType::BreaksDownFrom
                | TokenType::Major
                | TokenType::Minor
                | TokenType::Great
                | TokenType::Small
                | TokenType::UpperRight
                | TokenType::LowerRight
                | TokenType::UpperLeft
                | TokenType::LowerLeft
        )
    }

    /// Returns `true` if this token represents a relation of being or comparison.
    pub fn is_relational(&self) -> bool {
        matches!(
            self,
            TokenType::Is
                | TokenType::IsNot
                | TokenType::ToBeNot
                | TokenType::ToBeLike
                | TokenType::InvariantEquivalence
                | TokenType::LessThan
                | TokenType::GreaterThan
                | TokenType::LessEqual
                | TokenType::GreaterEqual
        )
    }

    /// Returns `true` if this token represents an arithmetic operator or keyword.
    pub fn is_arithmetic(&self) -> bool {
        matches!(self, TokenType::Plus | TokenType::Minus | TokenType::Times | TokenType::Div)
    }

    /// Identifies hardware locators (`::@`), supporting both dedicated [`TokenType::HardwareLocator`]
    /// and backward-compatible [`TokenType::And`] with lexeme `"::@"`.
    pub fn is_hardware_locator(&self, lexeme: &str) -> bool {
        matches!(self, TokenType::HardwareLocator)
            || (matches!(self, TokenType::And) && lexeme == "::@")
    }

    /// Checks if a string identifier represents a causal state-prime ($S \to S'$).
    pub fn is_state_prime_name(name: &str) -> bool {
        name.ends_with('`') || name.ends_with('\'')
    }

    /// Returns the stripped canonical base name without state-prime notation.
    pub fn strip_state_prime(name: &str) -> &str {
        if let Some(stripped) = name.strip_suffix('`') {
            stripped
```

```
        } else if let Some(stripped) = name.strip_suffix('\'') {
            stripped
        } else {
            name
        }
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Token Taxonomy Unit Proofs & Morphemic Invariant Tests )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_token_construction_and_eof() {
        let tok = Token::new(TokenType::Identifier("Character`".into()), "Character`", 1, 5);
        assert_eq!(tok.line, 1);
        assert_eq!(tok.col, 5);
        assert_eq!(tok.lexeme, "Character`");
        assert!(tok.is_state_prime());
        assert!(!tok.is_eof());

        let eof = Token::eof(10, 1);
        assert!(eof.is_eof());
    }

    #[test]
    fn test_hardware_locator_identification() {
        let dedicated = Token::new(TokenType::HardwareLocator, "::@", 1, 1);
        assert!(dedicated.is_hardware_locator());

        let legacy = Token::new(TokenType::And, "::@", 1, 1);
        assert!(legacy.is_hardware_locator());

        let regular_and = Token::new(TokenType::And, "::", 1, 1);
        assert!(!regular_and.is_hardware_locator());
    }

    #[test]
    fn test_state_prime_stripping() {
        assert_eq!(TokenType::strip_state_prime("Character`"), "Character");
        assert_eq!(TokenType::strip_state_prime("Cell'"), "Cell");
        assert_eq!(TokenType::strip_state_prime("Static"), "Static");
    }

    #[test]
    fn test_token_category_predicates() {
        assert!(TokenType::LBracket.is_open_boundary());
        assert!(TokenType::RBracket.is_close_boundary());
        assert!(TokenType::ActUpon.is_directional_vector());
        assert!(TokenType::InvariantEquivalence.is_relational());
        assert!(TokenType::Plus.is_arithmetic());
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/README.md MD 229 LINES 1814 WORDS 13.4 KB

```
# OmniCode AST (Abstract Syntax Tree) Subsystem

> *"And he gave unto Moses, when he had made an end of communing with him upon mount Sinai, two tables of testimony, tables of stone, written with the finger of God."*
> — **Exodus 31:18**
>
> *"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house."*
> — **Proverbs 24:27**
>
> *"Remove not the ancient landmark, which thy fathers have set."*
> — **Proverbs 22:28**
>
> *"He telleth the number of the stars; he calleth them all by their names."*
> — **Psalm 147:4**

---

## 1. Architectural Philosophy: The Structural Skeleton of OmniCode

In the OmniCode ecosystem, code is not an arbitrary string of characters or an impermanent script; it is a **covenantal inscription of structured thought**. Just as an organic body grows from an invariant skeletal scaffold before tissue, muscle, and skin are molded onto it ("fleshing out the bones"), the OmniCode compilation pipeline constructs an **Abstract Syntax Tree (AST)** that represents the immutable architecture of human-machine communication.

The AST subsystem in **Track 1 (Host Engine & Virtual Machine)** provides the unified, type-safe data model that bridges:

1. **The Parsing Pipeline (`src/parser/`)**: Ingests declarative source reels (`.omni`) and procedural process tapes (`.tape`).
2. **The Evaluator Runtime (`src/runtime/evaluator/`)**: Executes expressions, resolves causal vectors, verifies invariants, and manages transactional rollbacks.
3. **The Assembler & Transpiler (`src/runtime/assembler/`)**: Lowers high-level AST constructs into 64-bit balanced ternary machine words and transpiles them into standalone C and Lua targets.
4. **CPI-SI (Covenant Partnered Intelligence - Structural Intelligence)**: Embeds intelligent reasoning, contextual intent, and analogy-based linking directly into the syntax of the language.

---

## 2. Taxonomic Subsystem Organization

The AST subsystem is organized into seven modular domains under `src/ast/nodes/`, unified by the master enumeration `ASTNode` in `src/ast/nodes/mod.rs`:

```bash
src/ast/
├── mod.rs                      <-- Subsystem manifest and universal re-exports
├── README.md                   <-- Architectural manifesto and documentation
└── nodes/
    ├── mod.rs                  <-- Unified ASTNode enum and integration tests
    ├── boundaries.rs           <-- First-class asides, boundary frames, and topological casts
    ├── primitives.rs           <-- Scalar arithmetic, affix polarities, and unary inversions
    ├── relations.rs            <-- Relations of being, causal directional vectors, connectives
    ├── structural.rs           <-- Programs, modules, tapes, 4-block sections, rolls, commit gates
    ├── cardinal.rs             <-- Six Cardinal Interrogatives (WHO, WHAT, WHEN, WHERE, WHY, HOW)
    ├── operational.rs          <-- SVO Action Verbs, adversative guards, hardware locators
    └── ontological.rs          <-- Invariant types, entities, analogies, substrates, exemplars
```

---

3. The Seven Taxonomic Domains

Domain 1: Boundaries & First-Class Asides (`boundaries.rs`)

Boundaries are the first-class topological containers of OmniCode. Rather than using arbitrary parentheses or curly braces, every delimiter carries an ontological ranking:

Delimiter	Boundary Type	Semantics & Hardware Footprint
`[ ... ]`	**Denotation**	Physical allocation, concrete hardware address, fixed memory container.
`{ ... }`	**Connotation**	Behavioral context, dynamic execution scope, conceptual state.
`< ... >`	**Invariant**	Immutable covenant, compile-time verified truth, permanent constraint.
`{ ... ]`	**Top-Down**	Imposition of form onto physical matter (e.g. allocating memory).
`[ ... }`	**Bottom-Up**	Emergence of meaning from physical silicon (e.g. reading a sensor).
`( ... )`	**Aside**	Cognitive intent, machine reasoning, and epistemic commentary.
```

```
#### First-Class Cognitive Asides

Asides `( ... )` are not discarded comments! They are preserved in the AST (`AsideNode`), compiled into episodic memory scrolls (`.scroll`), and evaluated by the AI Partner (CPI-SI) to verify that the runtime actions align with the programmer's stated intent.

---

### Domain 2: Primitives & Polarity Affixes (`primitives.rs`)

Governs scalar arithmetic and the fundamental duality between abstract archetypes and concrete instances:

* **`ArithmeticNode`**: Binary mathematical operations (`+`, `-`, `*`, `/`). Division by zero is trapped as `QuadState::Halt` (NULL), preserving balanced equilibrium without crashing.
* **`UnaryNotNode`**: Algebraic truth inversion (`! expr` or `-- expr`).
* **`PolarityNode`**: Attaches an abstract (`+`) or concrete (`-`) bias to an identifier:
  * `+Cell`: The abstract blueprint or type specification of a Cell (zero-cost allocation).
  * `-Cell`: A physical, allocated, memory-backed instance of a Cell in silicon.
* **`BoundaryCastNode`**: Explicit transmutations between boundary domains (e.g. `[+]` or `{-}`).

---

### Domain 3: Relations, Directional Vectors & Connectives (`relations.rs`)

Governs the 3D computational coordinate system and formal logical connectives:

#### The Relational Matrix (`RelationNode`)

Evaluates state, identity, and similarity along the Ontological Y-Axis:

* **`=` (`BeingRelation::Is`)**: Exact identity or equivalence ($A = B$).
* **`!=` (`BeingRelation::IsNot`)**: Ontological distinction and inequality ("not to be").
* **`=!` (`BeingRelation::ToBeNot`)**: State predicate denial and negation ("is not" / to be not).
* **`=~` / `~=` (`BeingRelation::ToBeLike`)**: Structural analogy, homomorphism, or duck typing ("liken to").
* **`<=>` (`BeingRelation::Equivalence`)**: Bijective transduction across dual substrates (e.g. `65 <=> [ -1, +1, +1, -1, +1 ]`).
* **`<`, `>`, `<=`, `>=`**: Bounded comparisons across contexts and cell limits.

#### The Causal Directional Vectors (`DirectionalNode`)

Drives causal energy and data flow across time, scale, and spatial dimensions:

* **`-->` / `->` (`DirectionalVector::ActUpon`)**: Forward temporal causation and pipeline emission.
* **`<--` / `<-` (`DirectionalVector::ActAgain`)**: Retroactive feedback, retry, and transaction rollback.
* **`--^` / `-^` (`DirectionalVector::BuildsTo`)**: Ascending synthesis and architectural abstraction.
* **`--v` / `-v` (`DirectionalVector::BreaksDownFrom`)**: Descending decomposition and compiler lowering to IR.

#### The Connective Matrix (`ConnectiveNode`)

* **`::` (`Connective::And`)**: Cumulative simultaneous conjunction ($A \land B$).
* **`!` (`Connective::But`)**: Adversative caveat gate. If caveat $B$ fails, transaction $A$ is automatically rolled back.
* **`|` (`Connective::Or`)**: Disjunction with short-circuit evaluation.
* **`--` (`Connective::Not`)**: Balanced ternary truth inversion.
* **`:` (`Connective::InRelationTo`)**: Role classification and module import resolution.

---

### Domain 4: Structural Architecture (`structural.rs`)

Enforces the canonical 4-Block file architecture across the ecosystem:

* **`ProgramNode`**: Complete executable reel or tape stream.
* **`ModuleNode`**: Root module header declarations in `.omni` source reels.
* **`TapeNode`**: Process tape declarations in `.tape` operational files.
* **`SectionNode`**: The four canonical architectural blocks:
  1. **Block 1 (Metadata)**: Invariant declarations, author identity, and scripture witness.
  2. **Block 2 (Opening)**: Memory mounting, hardware locators, and bridge configurations.
  3. **Block 3 (Body)**: Sequential operational rolls driving state transformation.
  4. **Block 4 (Closing)**: Verification seals, tripwire tests, and commit gates.
* **`RollNode`**: An atomic procedural transaction: `[ ROLL N ] : [ Intent ] : Action ( Aside )`.
* **`CommitStatusNode`**: The closing gate asserting fourfold equilibrium (`PROCEED`, `AWAIT`, `ROLLBACK`, `HALT`).

---

### Domain 5: Cardinal Interrogatives (`cardinal.rs`)
```

Enables natural machine-operator dialogue and process tape headers through the Six Cardinal Interrogatives (`CardinalClauseNode`):

1. **`WHO`**: The Agent, Subject, Authority, or Module initiating the work.
2. **`WHAT`**: The Direct Object, Substance, or Data Payload being transformed.
3. **`WHEN`**: The Temporal Trigger, Clock Tick, or Execution Precondition.
4. **`WHERE`**: The Spatial Domain, Hardware Address, or Cell Space partition.
5. **`WHY`**: The Teleological Intent, Machine Reasoning, or Invariant Covenant.
6. **`HOW`**: The Modal Vector, Execution Status, or Velocity (+1, 0, NULL, -1).

Domain 6: Operational SVO Verbs & Guards (`operational.rs`)

Models active computation as Subject-Verb-Object (SVO) sentences:

* **`VerbNode`**: Action execution: `Subject --> VERB Object = Target`:
  * **`INSCRIBE`**: Writes an external or source value into a target cell.
  * **`TRANSDUCE`**: Bijectively translates values across dual substrates (bits $\iff$ trits).
  * **`GUARD`**: Asserts invariant equilibrium under adversative caveats.
  * **`PROJECT`**: Emits internal memory state to an external display, terminal, or port.
  * **`SEQUENCE`**: Chains discrete atomic cells into a continuous text ribbon.
* **`GuardNode`**: Adversative checkpoint asserting state integrity before advancing.
* **`HardwareLocatorNode`**: MMIO bus binding: `Target ::@ Address` (e.g. `[ Terminal Stdout ] ::@ 0x01`).

Domain 7: Ontological Typology, Analogies & Substrates (`ontological.rs`)

Defines the mathematical nature of types and memory substrates:

* **`InvariantTypeNode`**: Declarative definition of eternal types (e.g. `< Character >`, `< Number >`).
* **`EntityNode`**: Concrete typed instance bounded by an invariant type and physical substrate.
* **`AnalogyNode`**: Four-term proportional analogy (`A : B :: C : D`), the foundation of CPI-SI relational reasoning.
* **`SubstrateNode`**: Dual-substrate definition pairing physical binary silicon bits with virtual balanced ternary trits.
* **`ExemplarNode`**: Canonical mapping between integer scalars and balanced trits (`65 <=> [ -1, +1, +1, -1, +1 ]`). Includes automated verification via Horner power expansion.

---

4. The Dual-Rail Balanced Ternary Substrate

A cornerstone of OmniCode is the **dual-rail balanced ternary substrate**. In physical silicon, every virtual trit requires two binary physical rails (`pos_rail` and `neg_rail`) to encode four discrete truth states:

$$\mathbb{T}_4 = \{ \text{Await} (0), \text{Proceed} (+1), \text{Rollback} (-1), \text{Halt} (\text{NULL}) \}$$

```text
Rail Bit-Plane Layout (2 bits per virtual trit):
  pos_rail bit:  0   1   0   1
  neg_rail bit:  0   0   1   1
  ----------------------------
  Encoded State: 00  01  10  11
  Semantics:     0   +1  -1  NULL / Collision Trap
```

Key Mathematical Properties in the AST Subsystem

1. **Zero-Cost SIMD Rail-Swap**: Inverting a balanced ternary state or register requires no arithmetic ALU operations; it is a single SIMD register swap between the positive and negative bit-planes:
   $$\text{Swap}(P^+, P^-) \implies +1 \longleftrightarrow -1, \quad 0 \longleftrightarrow 0, \quad \text{NULL} \longleftrightarrow \text{NULL}$$
2. **Instant Hardware Collision Detection**: If both positive and negative rails are asserted simultaneously (`pos & neg != 0`), the hardware immediately traps the collision as a `NULL` fault, preventing illegal memory corruption.

---

5. State vs. Literal Disambiguation

The AST strictly distinguishes between **Equilibrium States** and **Literal Numeric Values**:

* **Equilibrium State (`0:AWAIT`)**: Represents dynamic neutrality, asynchronous standby, or an idling execution thread. Represented in the AST as `ASTNode::Ternary(QuadState::Await)`.
* **Air-Quoted Literal Value (`"0"` or `{ 0 }`)**: Represents the discrete scalar number zero (0.0). Represented in the AST as `ASTNode::Number(0.0)` or enclosed within a boundary.
* **Atomic Glyph (`'0'`)**: Represents the discrete 1-Cell character rune (ASCII 48 / trits `[0, 1, -1, -1, 1]`).

```
---

## 6. Readiness for the Next Modularization Phase

With the completion of the `universal/`, `character/`, and `number/` type modularizations in Track 2, the AST subsystem is now fully equipped with:

* Support for the complete connective and relation matrix (`::`, `!`, `|`, `--`, `!=`, `=!`, `=~`, `:`).
* First-class dual-rail substrate modeling and packing topologies in `SubstrateNode`.
* Horner-polynomial verification and zero-sum balance tracking in `ExemplarNode`.
* Reversible state kinematics and rail-swap negation in `QuadState`.

### Why We Hold Off on the Container Subsystem

The `container` type (which defines storehouses, arrays, queues, and tree structures) is an **aggregate vessel**. In accordance with Proverbs 24:27 (*"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house"*), a vessel cannot be soundly constructed until every atomic constituent it holds is fully proven:

1. **Universal Substrate**: Dual-rail balanced ternary foundation (*Proven*).
2. **Character**: Atomic 1-Cell glyph (*Proven*).
3. **Number**: Atomic 1-Word scalar (*Proven*).
4. **State / Text / Site**: Primitive execution states, ribbons of characters, and spatial addresses (*Next Phase*).
5. **Container**: The overarching vessel holding all of the above (*Final Aggregation Phase*).

This systematic modularization ensures zero circular dependencies and guarantees that each layer of the language rests upon an unshakeable bedrock of proven invariants.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/mod.rs RS 30 LINES 102 WORDS 1.1 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( AST Subsystem Identity and Export Manifest )
// ============================================================================
//! OmniCode AST Subsystem
//! Houses all syntax tree nodes, boundary containers, and expression shapes.

pub mod nodes;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for AST Types )
// ============================================================================
pub use nodes::*;


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( AST Subsystem Manifest Definitions )
// ============================================================================
// AST definitions are actively mapped in nodes.rs.


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Verification & Module Status )
// ============================================================================
// AST subsystem verified and sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/nodes/boundaries.rs RS 108 LINES 408 WORDS 3.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( AST Boundary Nodes: Containers, Typology & Intent Asides )
// ============================================================================
// SCRIPTURE ANCHORS & EPISTEMIC WITNESSES:
//   - Exodus 31:18: "written with the finger of God" — Inscription of eternal truth.
//   - Proverbs 24:27: "Prepare thy work without, and make it fit for thyself in the field;
//                      and afterwards build thine house." — Systematic modularization.
//   - Proverbs 22:28: "Remove not the ancient landmark, which thy fathers have set."
//                      — Preservation of invariant boundaries and balanced truth.
// ============================================================================

use crate::ast::nodes::ASTNode;
use crate::core::boundaries::BoundaryType;
use crate::core::operators::Polarity;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( First-Class Asides, Boundary Nodes & Boundary Casts )
// ============================================================================

/// First-class cognitive intent and machine reasoning aside `( ... )`.
#[derive(Debug, Clone, PartialEq)]
pub struct AsideNode {
    pub intent: String,
    pub line: usize,
    pub col: usize,
}

/// Boundary node enclosing child elements within a distinct topological frame:
/// `[...]` (Denotation), `{...}` (Connotation), `<...>` (Invariant),
/// `{...]` (Top-Down Imposition), `[...}` (Bottom-Up Emergence).
#[derive(Debug, Clone, PartialEq)]
pub struct BoundaryNode {
    pub boundary_type: BoundaryType,
    pub elements: Vec<ASTNode>,
    pub aside: Option<AsideNode>,
    pub line: usize,
    pub col: usize,
}

/// Boundary cast operator transmuting polarity or boundary topology.
#[derive(Debug, Clone, PartialEq)]
pub struct BoundaryCastNode {
    pub boundary: BoundaryType,
    pub polarity: Polarity,
    pub line: usize,
    pub col: usize,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Boundary Constructors, Predicates & Element Accessors )
// ============================================================================

impl AsideNode {
    /// Constructs a new [`AsideNode`].
    pub fn new(intent: impl Into<String>, line: usize, col: usize) -> Self {
        Self {
            intent: intent.into(),
            line,
            col,
        }
    }
}

impl BoundaryNode {
    /// Constructs a new [`BoundaryNode`] with no attached aside.
    pub fn new(boundary_type: BoundaryType, elements: Vec<ASTNode>, line: usize, col: usize) -> Self {
        Self {
            boundary_type,
            elements,
            aside: None,
            line,
```

```
            col,
        }
    }

    /// Attaches a first-class cognitive intent aside to the boundary.
    pub fn with_aside(mut self, aside: AsideNode) -> Self {
        self.aside = Some(aside);
        self
    }

    /// Returns `true` if this boundary is an immutable invariant `< ... >`.
    pub fn is_invariant(&self) -> bool {
        self.boundary_type.is_immutable()
    }
}

impl BoundaryCastNode {
    /// Constructs a new [`BoundaryCastNode`].
    pub fn new(boundary: BoundaryType, polarity: Polarity, line: usize, col: usize) -> Self {
        Self {
            boundary,
            polarity,
            line,
            col,
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Invariant Guarantees & Trait Implementations )
// ============================================================================
// Boundary nodes satisfy Send, Sync, Clone, PartialEq across all substrates.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/nodes/cardinal.rs RS 96 LINES 420 WORDS 3.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( AST Cardinal Nodes: Six Cardinal Interrogatives for Operator Dialogue )
// ============================================================================
// SCRIPTURE ANCHORS & EPISTEMIC WITNESSES:
//   - Exodus 31:18: "written with the finger of God" — Inscription of eternal truth.
//   - Proverbs 24:27: "Prepare thy work without, and make it fit for thyself in the field;
//                      and afterwards build thine house." — Systematic modularization.
//   - Proverbs 22:28: "Remove not the ancient landmark, which thy fathers have set."
//                      — Preservation of invariant boundaries and balanced truth.
// ============================================================================

use crate::ast::nodes::ASTNode;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( The 6 Cardinal Interrogatives & Cardinal Clause Definitions )
// ============================================================================

/// The Six Cardinal Interrogatives governing OmniCode process tapes and machine-operator dialog.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum CardinalKind {
    /// WHO: The Agent, Subject, or Authority responsible for the transaction.
    Who,
    /// WHAT: The Direct Object, Inscription Target, or Data Payload.
    What,
    /// WHEN: The Trigger, Event, Clock Tick, or Temporal Precondition.
    When,
    /// WHERE: The Container, Spatial Domain, Memory Address, or Cell Space.
    Where,
    /// WHY: The Teleological Reason, Machine Intent, or Invariant Covenant.
    Why,
    /// HOW: The Modal Vector, Execution Status, or Velocity (+1, 0, NULL, -1).
    How,
}

impl CardinalKind {
    /// Returns the uppercase string keyword for the interrogative.
    pub fn keyword(&self) -> &'static str {
        match self {
            Self::Who => "WHO",
            Self::What => "WHAT",
            Self::When => "WHEN",
            Self::Where => "WHERE",
            Self::Why => "WHY",
            Self::How => "HOW",
        }
    }

    /// Parses a string into a [`CardinalKind`] if matching one of the 6 Interrogatives.
    pub fn from_str_keyword(s: &str) -> Option<Self> {
        match s.to_ascii_uppercase().as_str() {
            "WHO" => Some(Self::Who),
            "WHAT" => Some(Self::What),
            "WHEN" => Some(Self::When),
            "WHERE" => Some(Self::Where),
            "WHY" => Some(Self::Why),
            "HOW" => Some(Self::How),
            _ => None,
        }
    }
}

/// A Cardinal Clause node inside a process tape header or dialogue session:
/// `CARDINAL { Payload } ::` (e.g. `WHO { OmniCode Lexicon } ::`).
#[derive(Debug, Clone, PartialEq)]
pub struct CardinalClauseNode {
    pub kind: CardinalKind,
    pub payload: Box<ASTNode>,
    pub line: usize,
    pub col: usize,
}

// ============================================================================
```

```
// BLOCK 3: BODY BLOCK
// ( Cardinal Constructors & Accessors )
// ================================================================================

impl CardinalClauseNode {
    /// Constructs a new [`CardinalClauseNode`].
    pub fn new(kind: CardinalKind, payload: ASTNode, line: usize, col: usize) -> Self {
        Self {
            kind,
            payload: Box::new(payload),
            line,
            col,
        }
    }
}

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Invariant Guarantees & Trait Implementations )
// ================================================================================
// Cardinal nodes satisfy Send, Sync, Clone, PartialEq across all targets.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/nodes/mod.rs RS 328 LINES 765 WORDS 11.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( AST Unified Node Subsystem: Complete Taxonomic Hierarchy )
// ============================================================================
// SCRIPTURE ANCHORS & EPISTEMIC WITNESSES:
//   - Exodus 31:18: "written with the finger of God" — Inscription of eternal truth.
//   - Proverbs 24:27: "Prepare thy work without, and make it fit for thyself in the field;
//                      and afterwards build thine house." — Systematic modularization.
//   - Proverbs 22:28: "Remove not the ancient landmark, which thy fathers have set."
//                      — Preservation of invariant boundaries and balanced truth.
//
// SUB-MODULE TAXONOMY:
//   - boundaries: First-class asides, boundary frames, and topological casts.
//   - primitives: Scalar arithmetic, affix polarity modifiers, and unary inversions.
//   - relations:  Ontological being relations, directional causal vectors, connectives.
//   - structural: Program reels, Module headers, Process tapes, 4-Block sections, Rolls, Commit gates.
//   - cardinal:   Six Cardinal Interrogatives (WHO, WHAT, WHEN, WHERE, WHY, HOW).
//   - operational: SVO Action Verbs (INSCRIBE, TRANSDUCE, GUARD, PROJECT, SEQUENCE), MMIO locators.
//   - ontological: Invariant type declarations, entities, analogies, substrates, exemplars.
// ============================================================================

pub mod boundaries;
pub mod primitives;
pub mod relations;
pub mod structural;
pub mod cardinal;
pub mod operational;
pub mod ontological;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Universal Re-Exports Across All Taxonomic Sub-Modules )
// ============================================================================

pub use boundaries::*;
pub use primitives::*;
pub use relations::*;
pub use structural::*;
pub use cardinal::*;
pub use operational::*;
pub use ontological::*;

use crate::core::ternary::QuadState;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Unified ASTNode Enumeration: Complete Multi-Substrate Node Hierarchy )
// ============================================================================

/// Unified AST Node enumeration representing every syntactic, structural,
/// relational, and operational construct in the OmniCode language.
#[derive(Debug, Clone, PartialEq)]
pub enum ASTNode {
    // ------------------------------------------------------------------------
    // 1. First-Class Asides, Literals & Primitives
    // ------------------------------------------------------------------------
    Aside(AsideNode),
    Ternary(QuadState),
    Identifier(String),
    Polarity(PolarityNode),
    BoundaryCast(BoundaryCastNode),
    Number(f64),
    StringLit(String),

    // ------------------------------------------------------------------------
    // 2. Boundary Containers & Concrete Entities
    // ------------------------------------------------------------------------
    Boundary(BoundaryNode),
    Entity(EntityNode),

    // ------------------------------------------------------------------------
    // 3. Relational Algebra, Vectors & Logic
    // ------------------------------------------------------------------------
    Relation(RelationNode),
```

```
    Directional(DirectionalNode),
    Connective(ConnectiveNode),
    Arithmetic(ArithmeticNode),
    UnaryNot(UnaryNotNode),

    // ------------------------------------------------------------------------
    // 4. Macrocosmic Program & 4-Block Section Structure
    // ------------------------------------------------------------------------
    Program(ProgramNode),
    Module(ModuleNode),
    Tape(TapeNode),
    Section(SectionNode),
    Roll(RollNode),
    CommitStatus(CommitStatusNode),

    // ------------------------------------------------------------------------
    // 5. Cardinal Interrogatives (Process Tape Clauses & Dialog)
    // ------------------------------------------------------------------------
    Cardinal(CardinalClauseNode),

    // ------------------------------------------------------------------------
    // 6. Operational SVO Verbs, Adversative Guards & Hardware Locators
    // ------------------------------------------------------------------------
    Verb(VerbNode),
    Guard(GuardNode),
    Hardware(HardwareLocatorNode),

    // ------------------------------------------------------------------------
    // 7. Invariant Ontological Typology, Analogies & Exemplars
    // ------------------------------------------------------------------------
    InvariantType(InvariantTypeNode),
    Analogy(AnalogyNode),
    Substrate(SubstrateNode),
    Exemplar(ExemplarNode),
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Trait Implementations, Send/Sync Invariant Verification & Unit Tests )
// ============================================================================
// All ASTNode variants implement Send, Sync, Clone, PartialEq.

#[cfg(test)]
mod tests {
    use super::*;
    use crate::core::boundaries::BoundaryType;
    use crate::core::operators::{BeingRelation, Connective, DirectionalVector, Polarity, AffixPosition};

    #[test]
    fn test_boundary_nodes_creation_and_asides() {
        let aside = AsideNode::new("Invariant scripture witness", 1, 1);
        assert_eq!(aside.intent, "Invariant scripture witness");

        let bound = BoundaryNode::new(BoundaryType::Denotation, vec![ASTNode::Identifier("Cell".to_string())], 2, 5)
            .with_aside(aside.clone());
        assert_eq!(bound.boundary_type, BoundaryType::Denotation);
        assert_eq!(bound.elements.len(), 1);
        assert_eq!(bound.aside, Some(aside));
    }

    #[test]
    fn test_structural_nodes_sections_rolls_tapes() {
        let roll = RollNode::new(
            1,
            "Inscribe",
            ASTNode::Identifier("ActionPayload".to_string()),
            10,
            1,
        ).with_aside(AsideNode::new("Roll 1 intent", 10, 20));

        assert_eq!(roll.id, 1);
        assert_eq!(roll.label, "Inscribe");
        assert!(roll.aside.is_some());

        let section = SectionNode::new(
            SectionKind::Body,
            vec![ASTNode::Roll(roll)],
            9,
```

```
            1,
        );
        assert_eq!(section.kind, SectionKind::Body);
        assert_eq!(section.elements.len(), 1);

        let tape = TapeNode::new("Character", 1, 1);
        assert_eq!(tape.name, "Character");

        let module = ModuleNode::new("Lexicon", 1, 1);
        assert_eq!(module.name, "Lexicon");
    }

    #[test]
    fn test_cardinal_nodes_interrogatives() {
        let who = CardinalClauseNode::new(
            CardinalKind::Who,
            ASTNode::Identifier("OmniCode Lexicon".to_string()),
            12,
            5,
        );
        assert_eq!(who.kind.keyword(), "WHO");
        assert_eq!(CardinalKind::from_str_keyword("what"), Some(CardinalKind::What));
        assert_eq!(CardinalKind::from_str_keyword("WHEN"), Some(CardinalKind::When));
        assert_eq!(CardinalKind::from_str_keyword("where"), Some(CardinalKind::Where));
        assert_eq!(CardinalKind::from_str_keyword("WHY"), Some(CardinalKind::Why));
        assert_eq!(CardinalKind::from_str_keyword("how"), Some(CardinalKind::How));
    }

    #[test]
    fn test_operational_verbs_and_guards() {
        let verb = VerbNode::new(
            VerbKind::Inscribe,
            Some(ASTNode::Identifier("Glyph".to_string())),
            ASTNode::Identifier("Cell".to_string()),
            Some(ASTNode::Identifier("Character`".to_string())),
            29,
            1,
        );
        assert_eq!(verb.verb.name(), "INSCRIBE");

        let guard = GuardNode::new(
            ASTNode::Ternary(QuadState::Proceed),
            ASTNode::Identifier("ProceedAction".to_string()),
            Some(ASTNode::Identifier("RollbackAction".to_string())),
            45,
            1,
        );
        assert_eq!(*guard.condition, ASTNode::Ternary(QuadState::Proceed));

        let hw = HardwareLocatorNode::new(
            ASTNode::Identifier("Terminal Stdout".to_string()),
            0x01,
            71,
            1,
        );
        assert_eq!(hw.address, 0x01);
    }

    #[test]
    fn test_ontological_invariant_types_and_exemplars() {
        let inv = InvariantTypeNode::new(
            "Character",
            ASTNode::Identifier("Idea { Type }".to_string()),
            25,
            1,
        );
        assert_eq!(inv.name, "Character");

        let exemplar = ExemplarNode::new(
            Some("Exemplar 'A'".to_string()),
            65,
            vec![-1, 1, 1, -1, 1],
            42,
            1,
        );
        assert_eq!(exemplar.scalar, 65);
        assert_eq!(exemplar.trits, vec![-1, 1, 1, -1, 1]);
        assert!(exemplar.verify_scalar());
```

```
        assert_eq!(exemplar.sum_trits(), 1);

        let substrate = SubstrateNode::new(8, 5, 33, 1)
            .with_topology("SymmetricInteger");
        assert_eq!(substrate.physical_bits, 8);
        assert_eq!(substrate.virtual_trits, 5);
        assert!(substrate.is_dual_rail());
        assert_eq!(substrate.topology(), Some("SymmetricInteger"));
    }

    #[test]
    fn test_primitives_and_relations() {
        let arith = ArithmeticNode::new(
            ASTNode::Number(1.0),
            ArithmeticOp::Add,
            ASTNode::Number(2.0),
            1,
            1,
        );
        assert_eq!(arith.op.symbol(), "+");

        let pol = PolarityNode::new(
            "proceed",
            Polarity::Abstract,
            AffixPosition::Prefix,
            1,
            1,
        );
        assert_eq!(pol.formatted(), "+proceed");

        let rel = RelationNode::new(
            ASTNode::Identifier("A".to_string()),
            BeingRelation::Is,
            ASTNode::Identifier("B".to_string()),
            1,
            1,
        );
        assert_eq!(rel.relation, BeingRelation::Is);
        assert!(rel.is_identity());

        let rel_not = RelationNode::new(
            ASTNode::Identifier("X".to_string()),
            BeingRelation::ToBeNot,
            ASTNode::Identifier("Y".to_string()),
            1,
            1,
        );
        assert!(rel_not.is_negation());

        let dir = DirectionalNode::new(
            ASTNode::Identifier("Source".to_string()),
            DirectionalVector::ActUpon,
            ASTNode::Identifier("Target".to_string()),
            1,
            1,
        );
        assert_eq!(dir.vector, DirectionalVector::ActUpon);

        let conn = ConnectiveNode::new(
            ASTNode::Identifier("Left".to_string()),
            Connective::And,
            ASTNode::Identifier("Right".to_string()),
            1,
            1,
        );
        assert_eq!(conn.connective, Connective::And);
        assert!(conn.is_conjunction());

        let conn_but = ConnectiveNode::new(
            ASTNode::Identifier("A".to_string()),
            Connective::But,
            ASTNode::Identifier("B".to_string()),
            1,
            1,
        );
        assert!(conn_but.is_adversative());
    }

```

```
    #[test]
    fn test_send_sync_invariants() {
        fn assert_send_sync<T: Send + Sync>() {}
        assert_send_sync::<ASTNode>();
        assert_send_sync::<AsideNode>();
        assert_send_sync::<BoundaryNode>();
        assert_send_sync::<ProgramNode>();
        assert_send_sync::<ModuleNode>();
        assert_send_sync::<TapeNode>();
        assert_send_sync::<SectionNode>();
        assert_send_sync::<RollNode>();
        assert_send_sync::<CardinalClauseNode>();
        assert_send_sync::<VerbNode>();
        assert_send_sync::<GuardNode>();
        assert_send_sync::<HardwareLocatorNode>();
        assert_send_sync::<InvariantTypeNode>();
        assert_send_sync::<ExemplarNode>();
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/nodes/ontological.rs RS 235 LINES 862 WORDS 7.2 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( AST Ontological Nodes: Invariant Types, Entities, Analogies & Exemplars )
// ================================================================================
// SCRIPTURE ANCHORS & EPISTEMIC WITNESSES:
//   - Exodus 31:18: "written with the finger of God" — Inscription of eternal truth.
//   - Proverbs 24:27: "Prepare thy work without, and make it fit for thyself in the field;
//                      and afterwards build thine house." — Systematic modularization.
//   - Proverbs 22:28: "Remove not the ancient landmark, which thy fathers have set."
//                      — Preservation of invariant boundaries and balanced truth.
// ================================================================================

use crate::ast::nodes::boundaries::{AsideNode, BoundaryNode};
use crate::ast::nodes::ASTNode;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Ontological Type, Entity & Epistemic Ratio Node Definitions )
// ================================================================================

/// Concrete typed entity allocated in memory:
/// `[ Name ]: < Invariant > & [ Substrate ] ( Aside ) { Context }`.
#[derive(Debug, Clone, PartialEq)]
pub struct EntityNode {
    pub name: String,
    pub definition: BoundaryNode,
    pub aside: Option<AsideNode>,
    pub context: Option<BoundaryNode>,
    pub line: usize,
    pub col: usize,
}

/// Freestanding Invariant Type definition:
/// `< Character > = < Idea { Type } > & [ Noun { Atomic Glyph } ] ( Aside ) { Invariants }`.
#[derive(Debug, Clone, PartialEq)]
pub struct InvariantTypeNode {
    pub name: String,
    pub category: Box<ASTNode>,
    pub aside: Option<AsideNode>,
    pub invariants: Vec<ASTNode>,
    pub line: usize,
    pub col: usize,
}

/// Ontological Proportion / Analogy node:
/// `[ Character ] : [ Text ] :: [ Trit ] : [ Word ]`.
#[derive(Debug, Clone, PartialEq)]
pub struct AnalogyNode {
    pub left_subject: Box<ASTNode>,
    pub left_context: Box<ASTNode>,
    pub right_subject: Box<ASTNode>,
    pub right_context: Box<ASTNode>,
    pub line: usize,
    pub col: usize,
}

/// Dual Substrate definition (Physical Silicon Bits & Virtual Balanced Trits):
/// `[ 1 Byte { 8 Bits } ] & [ 1 Cell { 5 Trits } ]` (Character: 1 Byte <=> 1 Cell)
/// `[ 8 Bytes { 64 Bits } ] & [ 1 Word { 27 Trits } ]` (Number: 8 Bytes <=> 1 Word).
#[derive(Debug, Clone, PartialEq)]
pub struct SubstrateNode {
    pub physical_bits: usize,
    pub virtual_trits: usize,
    pub is_dual_rail: bool,
    pub topology: Option<String>,
    pub line: usize,
    pub col: usize,
}

/// Canonical Exemplar mapping an integer scalar to balanced trits:
/// `65 <=> [ -1, +1, +1, -1, +1 ]` (Exemplar 'A').
#[derive(Debug, Clone, PartialEq)]
pub struct ExemplarNode {
    pub label: Option<String>,
```

```rust
    pub scalar: i64,
    pub trits: Vec<i8>,
    pub line: usize,
    pub col: usize,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Ontological Constructors & Accessors )
// ============================================================================

impl EntityNode {
    /// Constructs a new [`EntityNode`].
    pub fn new(name: impl Into<String>, definition: BoundaryNode, line: usize, col: usize) -> Self {
        Self {
            name: name.into(),
            definition,
            aside: None,
            context: None,
            line,
            col,
        }
    }

    /// Attaches an intent aside to this entity.
    pub fn with_aside(mut self, aside: AsideNode) -> Self {
        self.aside = Some(aside);
        self
    }

    /// Attaches an ontological context boundary to this entity.
    pub fn with_context(mut self, context: BoundaryNode) -> Self {
        self.context = Some(context);
        self
    }
}

impl InvariantTypeNode {
    /// Constructs a new [`InvariantTypeNode`].
    pub fn new(name: impl Into<String>, category: ASTNode, line: usize, col: usize) -> Self {
        Self {
            name: name.into(),
            category: Box::new(category),
            aside: None,
            invariants: Vec::new(),
            line,
            col,
        }
    }

    /// Attaches an intent aside to this invariant type.
    pub fn with_aside(mut self, aside: AsideNode) -> Self {
        self.aside = Some(aside);
        self
    }
}

impl AnalogyNode {
    /// Constructs a new 4-term ontological analogy proportion.
    pub fn new(
        left_subject: ASTNode,
        left_context: ASTNode,
        right_subject: ASTNode,
        right_context: ASTNode,
        line: usize,
        col: usize,
    ) -> Self {
        Self {
            left_subject: Box::new(left_subject),
            left_context: Box::new(left_context),
            right_subject: Box::new(right_subject),
            right_context: Box::new(right_context),
            line,
            col,
        }
    }
}

```

```rust
impl SubstrateNode {
    /// Constructs a new dual-substrate node with dual-rail enabled.
    pub fn new(
        physical_bits: usize,
        virtual_trits: usize,
        line: usize,
        col: usize,
    ) -> Self {
        Self {
            physical_bits,
            virtual_trits,
            is_dual_rail: true,
            topology: None,
            line,
            col,
        }
    }

    /// Configures the dual-rail balanced ternary mode.
    pub fn with_dual_rail(mut self, dual_rail: bool) -> Self {
        self.is_dual_rail = dual_rail;
        self
    }

    /// Attaches a radix-3 packing topology.
    pub fn with_topology(mut self, topology: impl Into<String>) -> Self {
        self.topology = Some(topology.into());
        self
    }

    /// Returns `true` if this substrate utilizes dual-rail silicon mapping.
    pub fn is_dual_rail(&self) -> bool {
        self.is_dual_rail
    }

    /// Returns the optional packing topology string slice.
    pub fn topology(&self) -> Option<&str> {
        self.topology.as_deref()
    }
}

impl ExemplarNode {
    /// Constructs a new canonical exemplar mapping.
    pub fn new(
        label: Option<String>,
        scalar: i64,
        trits: Vec<i8>,
        line: usize,
        col: usize,
    ) -> Self {
        Self {
            label,
            scalar,
            trits,
            line,
            col,
        }
    }

    /// Verifies that the Horner expansion of trits matches the scalar value:
    /// `sum_{i=0..N-1} trits[i] * 3^i == scalar`.
    pub fn verify_scalar(&self) -> bool {
        let mut total: i64 = 0;
        let mut multiplier: i64 = 1;
        for &t in &self.trits {
            total += (t as i64) * multiplier;
            multiplier *= 3;
        }
        total == self.scalar
    }

    /// Computes the net zero-sum balance of trits: `sum(trits)`.
    pub fn sum_trits(&self) -> i64 {
        self.trits.iter().map(|&t| t as i64).sum()
    }
}

// ============================================================================
```

```
// BLOCK 4: CLOSING BLOCK
// ( Invariant Guarantees & Trait Implementations )
// ================================================================================
// Ontological nodes satisfy Send, Sync, Clone, PartialEq across all targets.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/nodes/operational.rs RS | 158 LINES | 564 WORDS | 5.2 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( AST Operational Nodes: SVO Verbs, Adversative Guards & Hardware Locators )
// ============================================================================
// SCRIPTURE ANCHORS & EPISTEMIC WITNESSES:
//   - Exodus 31:18: "written with the finger of God" — Inscription of eternal truth.
//   - Proverbs 24:27: "Prepare thy work without, and make it fit for thyself in the field;
//                      and afterwards build thine house." — Systematic modularization.
//   - Proverbs 22:28: "Remove not the ancient landmark, which thy fathers have set."
//                      — Preservation of invariant boundaries and balanced truth.
// ============================================================================

use crate::ast::nodes::ASTNode;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Operational Action Node Definitions: SVO Verbs, Guards & Hardware Locators )
// ============================================================================

/// Canonical SVO Action Verbs in the OmniCode runtime grammar.
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub enum VerbKind {
    /// INSCRIBE: Imposes an external or source value into a target cell.
    Inscribe,
    /// TRANSDUCE: Maps values bijectively across dual substrates (bits <=> trits).
    Transduce,
    /// GUARD: Asserts invariant equilibrium under adversative caveats.
    Guard,
    /// PROJECT: Projects internal state into an external target buffer or screen.
    Project,
    /// SEQUENCE: Chains discrete cells outward into a continuous text ribbon.
    Sequence,
    /// User-defined or extensible domain action verb.
    Custom(String),
}

impl VerbKind {
    /// Parses an uppercase identifier into an action verb kind.
    pub fn from_name(name: &str) -> Self {
        match name.to_ascii_uppercase().as_str() {
            "INSCRIBE" => Self::Inscribe,
            "TRANSDUCE" => Self::Transduce,
            "GUARD" => Self::Guard,
            "PROJECT" => Self::Project,
            "SEQUENCE" => Self::Sequence,
            other => Self::Custom(other.to_string()),
        }
    }

    /// Returns the uppercase verb name string.
    pub fn name(&self) -> &str {
        match self {
            Self::Inscribe => "INSCRIBE",
            Self::Transduce => "TRANSDUCE",
            Self::Guard => "GUARD",
            Self::Project => "PROJECT",
            Self::Sequence => "SEQUENCE",
            Self::Custom(s) => s.as_str(),
        }
    }
}

/// SVO Action Verb execution node:
/// `Subject --> VERB Object = Target` (e.g. `{ Glyph ] --> INSCRIBE [ Cell ] = [ Character` ]`).
#[derive(Debug, Clone, PartialEq)]
pub struct VerbNode {
    pub verb: VerbKind,
    pub subject: Option<Box<ASTNode>>,
    pub object: Box<ASTNode>,
    pub target: Option<Box<ASTNode>>,
    pub line: usize,
    pub col: usize,
}

```

```rust
/// Adversative Guard assertion node:
/// `Condition ! GUARD --> ProceedBranch | FallbackBranch`.
#[derive(Debug, Clone, PartialEq)]
pub struct GuardNode {
    pub condition: Box<ASTNode>,
    pub on_proceed: Box<ASTNode>,
    pub on_fallback: Option<Box<ASTNode>>,
    pub line: usize,
    pub col: usize,
}

/// MMIO Hardware Bus locator node:
/// `Target ::@ Address` (e.g. `[ Terminal Stdout ] ::@ 0x01`).
#[derive(Debug, Clone, PartialEq)]
pub struct HardwareLocatorNode {
    pub target: Box<ASTNode>,
    pub address: u64,
    pub line: usize,
    pub col: usize,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Operational Constructors & Methods )
// ============================================================================

impl VerbNode {
    /// Constructs a new [`VerbNode`].
    pub fn new(
        verb: VerbKind,
        subject: Option<ASTNode>,
        object: ASTNode,
        target: Option<ASTNode>,
        line: usize,
        col: usize,
    ) -> Self {
        Self {
            verb,
            subject: subject.map(Box::new),
            object: Box::new(object),
            target: target.map(Box::new),
            line,
            col,
        }
    }
}

impl GuardNode {
    /// Constructs a new [`GuardNode`].
    pub fn new(
        condition: ASTNode,
        on_proceed: ASTNode,
        on_fallback: Option<ASTNode>,
        line: usize,
        col: usize,
    ) -> Self {
        Self {
            condition: Box::new(condition),
            on_proceed: Box::new(on_proceed),
            on_fallback: on_fallback.map(Box::new),
            line,
            col,
        }
    }
}

impl HardwareLocatorNode {
    /// Constructs a new [`HardwareLocatorNode`].
    pub fn new(target: ASTNode, address: u64, line: usize, col: usize) -> Self {
        Self {
            target: Box::new(target),
            address,
            line,
            col,
        }
    }
}

```

```
// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Invariant Guarantees & Trait Implementations )
// ================================================================================
// Operational nodes satisfy Send, Sync, Clone, PartialEq across all targets.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/nodes/primitives.rs RS 125 LINES 445 WORDS 4.1 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( AST Primitive Nodes: Arithmetic, Polarity & Unary Inversion )
// ============================================================================
// SCRIPTURE ANCHORS & EPISTEMIC WITNESSES:
//   - Exodus 31:18: "written with the finger of God" — Inscription of eternal truth.
//   - Proverbs 24:27: "Prepare thy work without, and make it fit for thyself in the field;
//                      and afterwards build thine house." — Systematic modularization.
//   - Proverbs 22:28: "Remove not the ancient landmark, which thy fathers have set."
//                      — Preservation of invariant boundaries and balanced truth.
// ============================================================================

use crate::ast::nodes::ASTNode;
use crate::core::operators::{AffixPosition, Polarity};

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Binary Arithmetic Operators, Arithmetic Nodes & Polarity Modifiers )
// ============================================================================

/// Discrete arithmetic operations supported on scalar values.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum ArithmeticOp {
    Add,
    Sub,
    Mul,
    Div,
}

impl ArithmeticOp {
    /// Returns the canonical mathematical operator symbol.
    pub fn symbol(&self) -> &'static str {
        match self {
            Self::Add => "+",
            Self::Sub => "-",
            Self::Mul => "*",
            Self::Div => "/",
        }
    }
}

/// Binary arithmetic evaluation node: `left OP right`.
#[derive(Debug, Clone, PartialEq)]
pub struct ArithmeticNode {
    pub left: Box<ASTNode>,
    pub op: ArithmeticOp,
    pub right: Box<ASTNode>,
    pub line: usize,
    pub col: usize,
}

/// Unary logical/ternary inversion node: `! operand`.
#[derive(Debug, Clone, PartialEq)]
pub struct UnaryNotNode {
    pub operand: Box<ASTNode>,
    pub line: usize,
    pub col: usize,
}

/// Affixed polarity modifier indicating abstract or concrete bias: `+name` or `-name`.
#[derive(Debug, Clone, PartialEq)]
pub struct PolarityNode {
    pub name: String,
    pub polarity: Polarity,
    pub position: AffixPosition,
    pub line: usize,
    pub col: usize,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Primitive Node Constructors & Operational Helpers )
// ============================================================================

```

```
impl ArithmeticNode {
    /// Constructs a new binary arithmetic node.
    pub fn new(left: ASTNode, op: ArithmeticOp, right: ASTNode, line: usize, col: usize) -> Self {
        Self {
            left: Box::new(left),
            op,
            right: Box::new(right),
            line,
            col,
        }
    }
}

impl UnaryNotNode {
    /// Constructs a new unary inversion node.
    pub fn new(operand: ASTNode, line: usize, col: usize) -> Self {
        Self {
            operand: Box::new(operand),
            line,
            col,
        }
    }
}

impl PolarityNode {
    /// Constructs a new polarity-modified entity node.
    pub fn new(name: impl Into<String>, polarity: Polarity, position: AffixPosition, line: usize, col: usize) -> Self {
        Self {
            name: name.into(),
            polarity,
            position,
            line,
            col,
        }
    }

    /// Returns the formatted symbol representation with its affix.
    pub fn formatted(&self) -> String {
        match self.position {
            AffixPosition::Prefix => format!("{}{}", self.polarity.symbol(), self.name),
            AffixPosition::Postfix => format!("{}{}", self.name, self.polarity.symbol()),
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Invariant Guarantees & Trait Implementations )
// ============================================================================
// Primitive nodes satisfy Send, Sync, Clone, PartialEq across all targets.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/nodes/relations.rs RS 169 LINES 594 WORDS 5.3 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( AST Relational Nodes: Being Relations, Directional Vectors & Connectives )
// ================================================================================
// SCRIPTURE ANCHORS & EPISTEMIC WITNESSES:
//   - Exodus 31:18: "written with the finger of God" — Inscription of eternal truth.
//   - Proverbs 24:27: "Prepare thy work without, and make it fit for thyself in the field;
//                      and afterwards build thine house." — Systematic modularization.
//   - Proverbs 22:28: "Remove not the ancient landmark, which thy fathers have set."
//                      — Preservation of invariant boundaries and balanced truth.
// ================================================================================

use crate::ast::nodes::ASTNode;
use crate::core::operators::{BeingRelation, Connective, DirectionalVector};

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Relational, Directional & Connective Structural Node Definitions )
// ================================================================================

/// Relational node evaluating ontological identity, state, equivalence:
/// `left RELATION right` (`=`, `!=`, `=!`, `~=`, `<=>`, `<`, `>`, `<=`, `>=`).
#[derive(Debug, Clone, PartialEq)]
pub struct RelationNode {
    pub left: Box<ASTNode>,
    pub relation: BeingRelation,
    pub right: Box<ASTNode>,
    pub line: usize,
    pub col: usize,
}

/// Directional causal propagation node driving energy and state across boundaries:
/// `source VECTOR target` (`-->`, `<--`, `--^`, `--v`, `->`, `<-`).
#[derive(Debug, Clone, PartialEq)]
pub struct DirectionalNode {
    pub source: Box<ASTNode>,
    pub vector: DirectionalVector,
    pub target: Box<ASTNode>,
    pub line: usize,
    pub col: usize,
}

/// Connective node conjoining or disjoining expressions through formal logic:
/// `left CONNECTIVE right` (`::` And, `!` But, `|` Or, `--` Not, `:` InRelationTo).
#[derive(Debug, Clone, PartialEq)]
pub struct ConnectiveNode {
    pub left: Box<ASTNode>,
    pub connective: Connective,
    pub right: Box<ASTNode>,
    pub line: usize,
    pub col: usize,
}

// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Relational Constructors & Query Helpers )
// ================================================================================

impl RelationNode {
    /// Constructs a new relational node.
    pub fn new(
        left: ASTNode,
        relation: BeingRelation,
        right: ASTNode,
        line: usize,
        col: usize,
    ) -> Self {
        Self {
            left: Box::new(left),
            relation,
            right: Box::new(right),
            line,
            col,
        }
```

```rust
    }

    /// Returns `true` if this is an identity relation (`=`).
    pub fn is_identity(&self) -> bool {
        self.relation == BeingRelation::Is
    }

    /// Returns `true` if this is a negative relation (`!=` or `=!`).
    pub fn is_negation(&self) -> bool {
        matches!(
            self.relation,
            BeingRelation::IsNot | BeingRelation::ToBeNot
        )
    }

    /// Returns `true` if this is an analogical likeness relation (`~=`).
    pub fn is_likeness(&self) -> bool {
        self.relation == BeingRelation::ToBeLike
    }

    /// Returns `true` if this is invariant equivalence (`<=>`).
    pub fn is_equivalence(&self) -> bool {
        self.relation == BeingRelation::Equivalence
    }
}

impl DirectionalNode {
    /// Constructs a new directional vector node.
    pub fn new(
        source: ASTNode,
        vector: DirectionalVector,
        target: ASTNode,
        line: usize,
        col: usize,
    ) -> Self {
        Self {
            source: Box::new(source),
            vector,
            target: Box::new(target),
            line,
            col,
        }
    }
}

impl ConnectiveNode {
    /// Constructs a new connective node.
    pub fn new(
        left: ASTNode,
        connective: Connective,
        right: ASTNode,
        line: usize,
        col: usize,
    ) -> Self {
        Self {
            left: Box::new(left),
            connective,
            right: Box::new(right),
            line,
            col,
        }
    }

    /// Returns `true` if this is an adversative caveat (`!`).
    pub fn is_adversative(&self) -> bool {
        self.connective == Connective::But
    }

    /// Returns `true` if this is conjunction (`::`).
    pub fn is_conjunction(&self) -> bool {
        self.connective == Connective::And
    }

    /// Returns `true` if this is disjunction (`|`).
    pub fn is_disjunction(&self) -> bool {
        self.connective == Connective::Or
    }

```

```
    /// Returns `true` if this is unary inversion (`--`).
    pub fn is_inversion(&self) -> bool {
        self.connective == Connective::Not
    }

    /// Returns `true` if this is role typing / relation (`:`).
    pub fn is_role(&self) -> bool {
        self.connective == Connective::InRelationTo
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Invariant Guarantees & Trait Implementations )
// ============================================================================
// Relational nodes satisfy Send, Sync, Clone, PartialEq across all targets.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ast/nodes/structural.rs RS 205 LINES 769 WORDS 6.3 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( AST Structural Nodes: Programs, Modules, Tapes, Sections, Rolls & Commit Gates )
// ============================================================================
// SCRIPTURE ANCHORS & EPISTEMIC WITNESSES:
//   - Exodus 31:18: "written with the finger of God" — Inscription of eternal truth.
//   - Proverbs 24:27: "Prepare thy work without, and make it fit for thyself in the field;
//                      and afterwards build thine house." — Systematic modularization.
//   - Proverbs 22:28: "Remove not the ancient landmark, which thy fathers have set."
//                      — Preservation of invariant boundaries and balanced truth.
// ============================================================================

use crate::ast::nodes::boundaries::AsideNode;
use crate::ast::nodes::ASTNode;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Structural Node Definitions & 4-Block Section Kinds )
// ============================================================================

/// Complete source program reel or process tape containing sequential statements.
#[derive(Debug, Clone, PartialEq)]
pub struct ProgramNode {
    pub statements: Vec<ASTNode>,
    pub line: usize,
    pub col: usize,
}

/// Root Module declaration: `= [Module]: < Category > & [ Noun ] ( Aside ) { Fields }`.
#[derive(Debug, Clone, PartialEq)]
pub struct ModuleNode {
    pub name: String,
    pub category: Option<Box<ASTNode>>,
    pub aside: Option<AsideNode>,
    pub fields: Vec<ASTNode>,
    pub line: usize,
    pub col: usize,
}

/// Root Process Tape declaration: `= [Tape]: < Category > & [ Process ] ( Aside ) { Clauses }`.
#[derive(Debug, Clone, PartialEq)]
pub struct TapeNode {
    pub name: String,
    pub category: Option<Box<ASTNode>>,
    pub aside: Option<AsideNode>,
    pub clauses: Vec<ASTNode>,
    pub line: usize,
    pub col: usize,
}

/// The four canonical blocks governing OmniCode file architecture.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum SectionKind {
    /// Block 1: Metadata / Invariant Declarations
    Metadata,
    /// Block 2: Opening Block — State mounting, allocations, hardware bridges
    Opening,
    /// Block 3: Body Block — Sequential operational rolls & causal state progression
    Body,
    /// Block 4: Closing Block — Verification, sealing, tripwiring, diagnostic log & commit gates
    Closing,
}

impl SectionKind {
    /// Returns the canonical section block name.
    pub fn name(&self) -> &'static str {
        match self {
            Self::Metadata => "Metadata",
            Self::Opening => "Opening",
            Self::Body => "Body",
            Self::Closing => "Closing",
        }
    }
}
```

```rust

/// A 4-Block section container holding child statements or operational rolls:
/// `Opening { ... }`, `Body [ ... ]`, `Closing ( Aside ) { ... }`.
#[derive(Debug, Clone, PartialEq)]
pub struct SectionNode {
    pub kind: SectionKind,
    pub aside: Option<AsideNode>,
    pub elements: Vec<ASTNode>,
    pub line: usize,
    pub col: usize,
}

/// An atomic operational roll within the Body block:
/// `[ ROLL N ] : [ Intent ] : Action ( Aside )`.
#[derive(Debug, Clone, PartialEq)]
pub struct RollNode {
    pub id: usize,
    pub label: String,
    pub action: Box<ASTNode>,
    pub aside: Option<AsideNode>,
    pub line: usize,
    pub col: usize,
}

/// Closing Block Fourfold Commit Status Gate:
/// `[ Commit Status ] : [ +1:PROCEED | 0:AWAIT | HALT:NULL | -1:ROLLBACK ]`.
#[derive(Debug, Clone, PartialEq)]
pub struct CommitStatusNode {
    pub cases: Vec<ASTNode>,
    pub aside: Option<AsideNode>,
    pub line: usize,
    pub col: usize,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Structural Constructors & Query Implementations )
// ============================================================================

impl ProgramNode {
    /// Constructs a new [`ProgramNode`].
    pub fn new(statements: Vec<ASTNode>, line: usize, col: usize) -> Self {
        Self { statements, line, col }
    }
}

impl ModuleNode {
    /// Constructs a new [`ModuleNode`].
    pub fn new(name: impl Into<String>, line: usize, col: usize) -> Self {
        Self {
            name: name.into(),
            category: None,
            aside: None,
            fields: Vec::new(),
            line,
            col,
        }
    }
}

impl TapeNode {
    /// Constructs a new [`TapeNode`].
    pub fn new(name: impl Into<String>, line: usize, col: usize) -> Self {
        Self {
            name: name.into(),
            category: None,
            aside: None,
            clauses: Vec::new(),
            line,
            col,
        }
    }
}

impl SectionNode {
    /// Constructs a new [`SectionNode`].
    pub fn new(kind: SectionKind, elements: Vec<ASTNode>, line: usize, col: usize) -> Self {
        Self {
```

```
            kind,
            aside: None,
            elements,
            line,
            col,
        }
    }

    /// Attaches an intent aside to this section block.
    pub fn with_aside(mut self, aside: AsideNode) -> Self {
        self.aside = Some(aside);
        self
    }
}

impl RollNode {
    /// Constructs a new operational [`RollNode`].
    pub fn new(id: usize, label: impl Into<String>, action: ASTNode, line: usize, col: usize) -> Self {
        Self {
            id,
            label: label.into(),
            action: Box::new(action),
            aside: None,
            line,
            col,
        }
    }

    /// Attaches an intent aside to this roll.
    pub fn with_aside(mut self, aside: AsideNode) -> Self {
        self.aside = Some(aside);
        self
    }
}

impl CommitStatusNode {
    /// Constructs a new [`CommitStatusNode`].
    pub fn new(cases: Vec<ASTNode>, line: usize, col: usize) -> Self {
        Self {
            cases,
            aside: None,
            line,
            col,
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Invariant Guarantees & Trait Implementations )
// ============================================================================
// Structural nodes satisfy Send, Sync, Clone, PartialEq across all targets.
```